



河南大學
HENAN UNIVERSITY

计算机操作系统

主讲人：胡 萍

明德，新民

止于至善



第四章 存储器管理

第1讲



本章内容

4.1 存储器的层次结构

4.2 程序的装入和链接

4.3 连续分配存储管理方式

4.4 对换(Swapping)

4.5 分页存储管理方式

4.6 分段存储管理方式





4.1 存储器的层次结构

□ 4.1.1 多层结构的存储器系统

□ 4.1.2 主存储器与寄存器

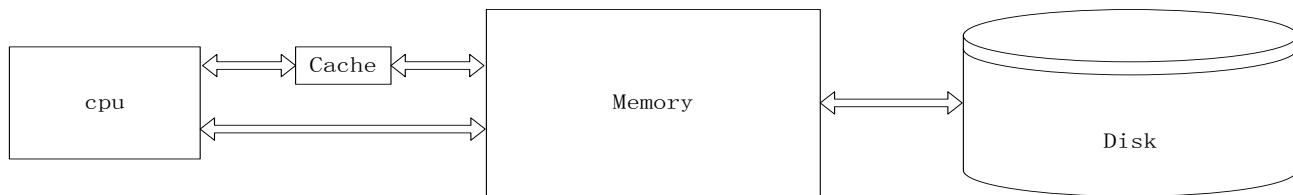
□ 4.1.3 高速缓存和磁盘缓存



4.1.1 多层结构的存储器系统

1. 存储器的多层结构

存储器是计算机系统的重要组成部分，通常包括外存、内存、Cache、寄存器等四个级别的设备。



本章针对的是内存的管理

2. 可执行存储器

在计算机系统的存储层次中，寄存器和主存储器又被称为可执行存储器。



4.1.2 主存储器与寄存器

1. 主存储器

主存储器简称内存或主存，是计算机系统中的主要部件，用于保存**进程运行时的程序和数据**，也称可执行存储器。通常，处理机都是从主存储器中取得指令和数据的。

2. 可执行存储器

寄存器具有与处理机相同的速度，故对寄存器的访问速度最快，完全能与**CPU**协调工作，但价格却十分昂贵，因此容量不可能做得很大。



4.1.3 高速缓存和磁盘缓存

1. 高速缓存

在计算机系统中，为了缓和**内存与处理机速度之间**的矛盾，许多地方都设置了缓存。

2. 磁盘缓存

为了缓和**磁盘和主存之间速度的不匹配**，设置了磁盘缓存，主要用于暂时存放频繁使用的一部分磁盘数据和信息，以减少访问磁盘的次数。

磁盘缓存本身**并不是一种实际存在**的存储器，而是利用主存中的部分存储空间暂时存放从磁盘中读出（或写入）的信息。



4.2 程序的装入和链接

□ 4.2.1 程序的装入

□ 4.2.2 程序的链接

- 逻辑地址：目标代码的相对编址
- 物理地址：内存存储单元的编址

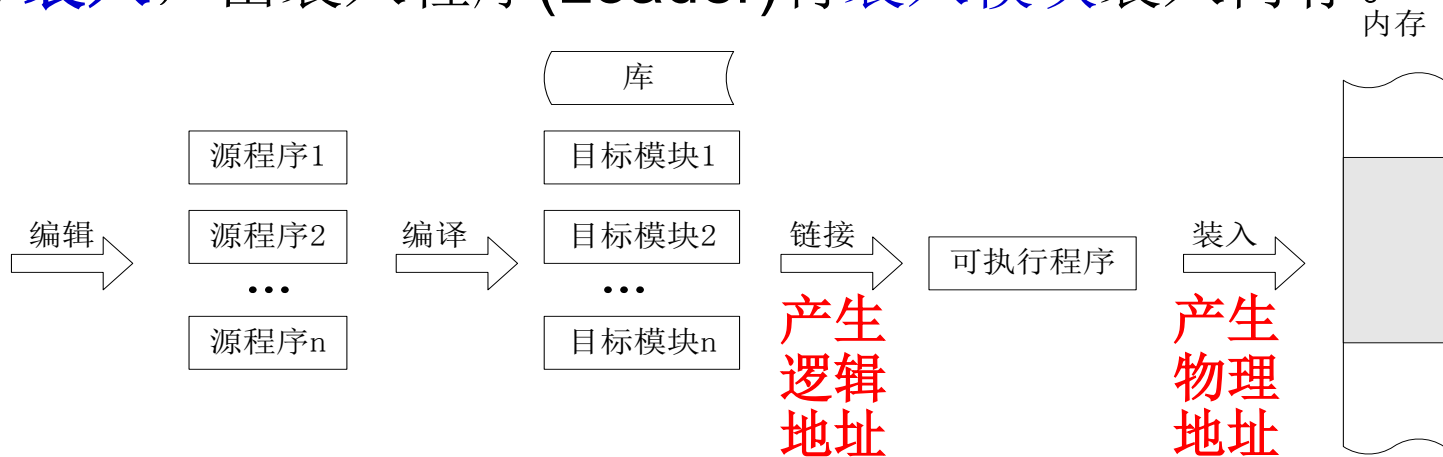
重定位，为何？



4.2 程序的装入和链接

用户程序要在系统中运行，通常都要经过以下三个步骤：

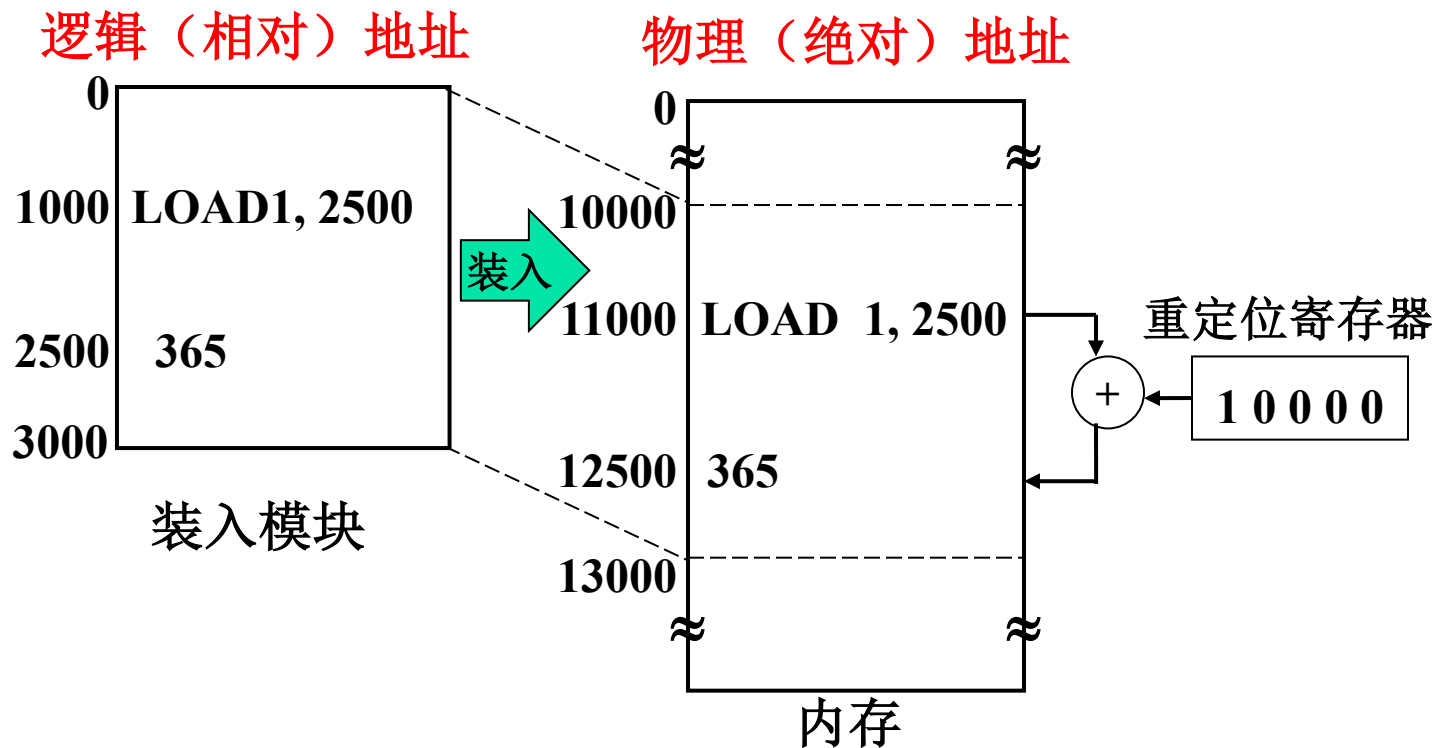
- (1) **编译**，由编译程序(Compiler)对用户源程序进行编译，形成若干个目标模块(Object Module)；**转换为机器语言**
- (2) **链接**，由链接程序(Linker)将编译后形成的一组目标模块以及它们所需要的库函数链接在一起，形成一个完整的装入模块(Load Module)；**转换为可执行程序**
- (3) **装入**，由装入程序(Loader)将**装入模块**装入内存。



- 逻辑地址：目标代码的相对编址
- 物理地址：内存存储单元的编址

重定位，为何？

4.2 程序的装入和链接



➔ 若不进行地址转换，将会发生运行错误。

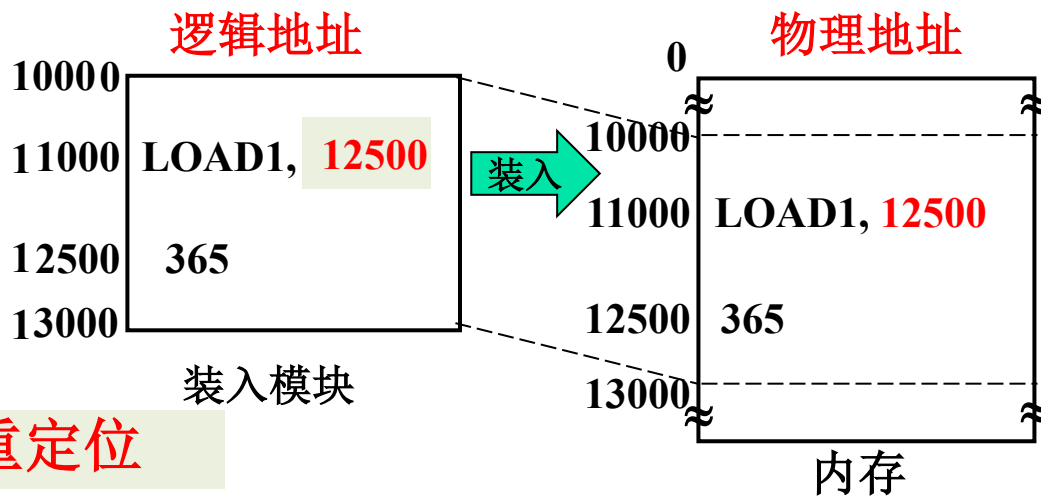


4.2.1 程序的装入

三种装入方式：绝对装入、可重定位装入和动态运行时装入。

1. 绝对装入方式(Absolute Loading Mode) 适用于单道程序环境

绝对装入方式根据程序在内存中将要驻留的起始地址，选择该地址作为目标模块的链接起始地址，产生与驻留内存物理地址一致的装入模块，即程序逻辑地址与内存物理地址完全相同。



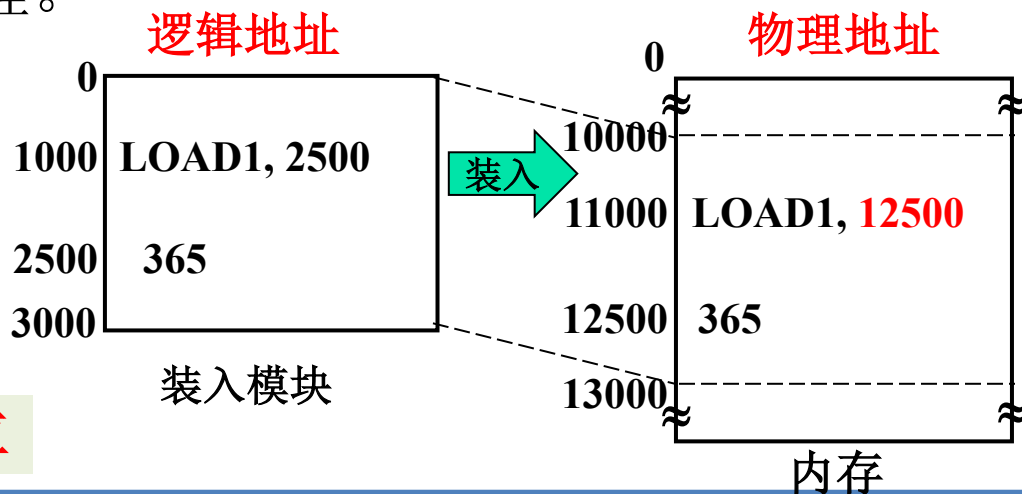
编译时完成重定位



4.2.1 程序的装入

2. 可重定位装入方式(Relocation Loading Mode)

- 根据**内存的具体情况**将装入模块装入到内存的适当位置。因为地址变换是在**装入时一次性完成**的，以后不会再改变，所以称为**静态重定位**。
- **重定位**：装入时，将装入模块中指令和数据的逻辑地址修改为物理地址的过程。



装入时时完成重定位



4.2.1 程序的装入

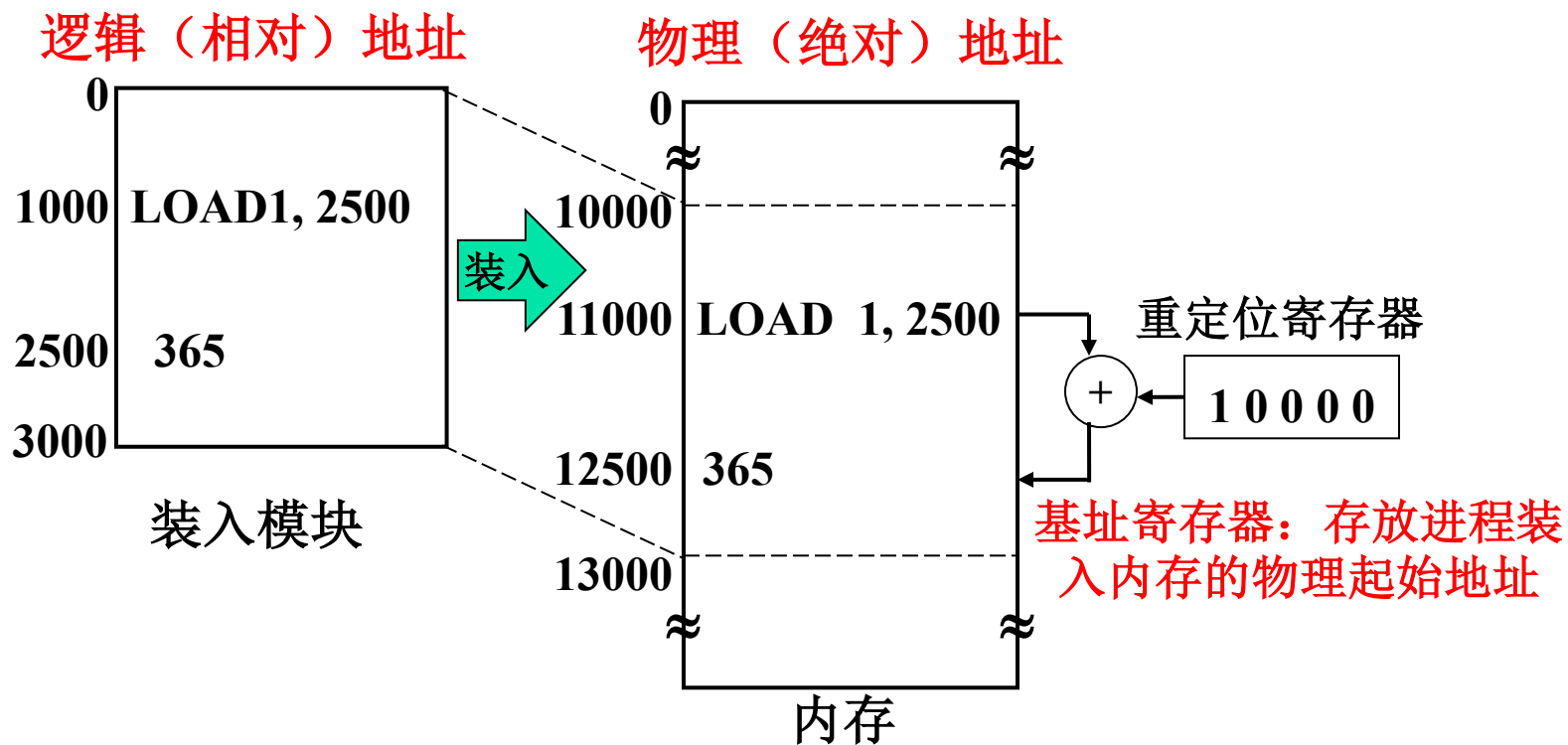
3.动态运行时的装入方式(Dynamic Run-time Loading)

- ❑ 可重定位装入方式可以将程序装入到内存的任何位置，但不允许在内存中移动位置。
- ❑ 动态运行时装入方式：在把装入模块装入内存时，并不立即把装入模块中的逻辑地址转换为绝对地址，而是把这种**地址转换推迟到程序真正运行时才进行**。因此装入内存后的所有地址都仍是**逻辑地址**。
- ❑ 采用动态运行时装入方式，程序运行时逻辑地址到内存物理地址的转换靠**硬件地址转换机构**实现，通常设置一个**重定位寄存器**，加载操作系统为程序分配的内存空间起始物理地址。



4.2.1 程序的装入

3. 动态运行时的装入方式(Dynamic Run-time Loading)



执行时完成重定位



4.2.1 程序的装入

程序的装入：

- **绝对装入**：目标代码的逻辑地址与装入的物理地址一致；
- **静态重定位装入**：一边装入，一边实现地址转换；
- **动态重定位装入**：原代码装入，待执行时再进行地址转换。



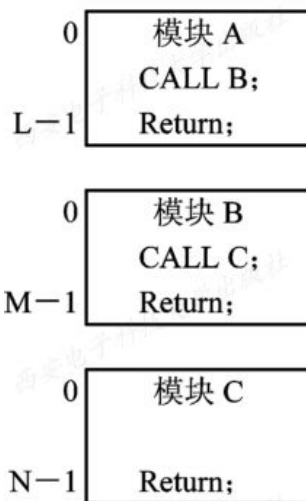
4.2.2 程序的链接

1. 静态链接(Static Linking)方式

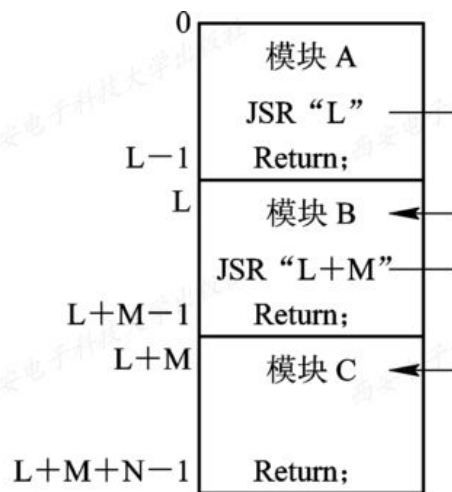
程序运行之前，多个目标模块及所需的库函数事先进行链接形成一个完整的装入模块（称为**可执行文件**）存储于外存，以后不再拆分。

必须解决两个问题：

- 修改目标模块的相对地址；
- 变换外部调用符号。



(a) 目标模块



(b) 装入模块



4.2.2 程序的链接

2. 装入时动态链接(Load-time Dynamic Linking)

目标模块装入内存时，采用**边装入边链接**的链接方式。即在装入一个目标模块时，若发生一个外部模块调用事件，将引起装入程序去找出相应的外部目标模块，并将它装入内存，并修改目标模块中的相对地址。

装入时动态链接方式有两个优点：

- 便于目标模块的修改和更新
- 有利于实现目标模块的共享



4.2.2 程序的链接

3. 运行时动态链接(Run-time Dynamic Linking)

运行时动态链接是将某些模块的链接推迟到程序执行时才进行。在程序执行中，当发现某个被调用目标模块尚未链接，立即由操作系统去找到该目标模块并将之装入内存，再把它链接到调用者模块上。

运行时动态链接方式有两个优点：

- 加快程序的装入过程
- 节省大量内存空间



● 存储分配（管理）方式

无论采用何种方式装入，均涉及到内存空间共享与分配问题。
分配方式有：

- 连续分配：为一个用户程序分配一个连续的内存空间，又称分区管理方式。
- 离散分配：为作业（进程）分配不连续存储空间

内部碎片： 分配给某进程的内存区域中，没有用上的部分。

外部碎片： 内存中，由于太小而难以利用的某些空闲分区。



● 存储分配（管理）方式

连续内存分配方式

4.3

■ 分区管理方式

单一连续分配
固定分区分配
动态分区分配
可重定位分区分配

离散内存分配方式

- 分页（页式）存储管理方式 4.5
- 分段（段式）存储管理方式 4.6
 - 段页式存储管理方式
- 虚拟存储器
 - 请求分页存储管理方式
 - 请求分段存储管理方式



4.3 连续分配存储管理方式

- 4.3.1 单一连续分配管理方式
- 4.3.2 固定分区分配
- 4.3.3 动态分区分配（可变分区分配）
- 4.3.4 基于顺序搜索的动态分区分配算法
- 4.3.5 基于索引搜索的动态分区分配算法
- 4.3.6 动态可重定位分区分配



4.3 连续分配存储管理方式

- 是指一个用户程序分配一个**连续的内存空间**。又称**分区管理方式**。分区是指内存中的一个连续区域。
- 分区分配方式可分为：**单一连续分配**、**固定分区分配**、**动态分区分配**、**动态重定位分区分配**等。

分区管理方式曾被广泛应用于20世纪60~70年代的OS中，至今仍在内存分配方式中占一席之地。

优点：实现简单，无外部碎片；

缺点：适用于单用户、单任务系统；有**内部碎片**；存储器利用低。



4.3.1 单一连续分配管理方式

- 单一连续分配管理方式：**将内存分为系统区和用户区。**
- 系统区提供给操作系统使用，它通常放在内存的**低址**部分。
- 用户区提供给用户程序使用，一次只能装入一个程序运行，即整个内存的用户空间由该程序独占。

适用于早期单用户
单任务OS。



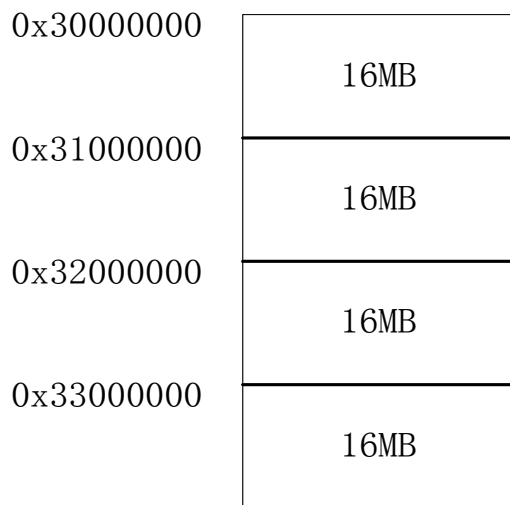


4.3.2 固定分区分配

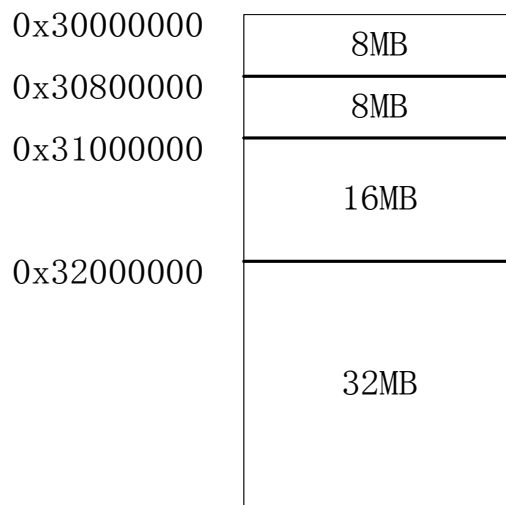
最简单的可运行**多道程序**的
存储器管理方式。

□ 思想：内存可用区划分成若干个大小固定的存区，每个存区分别装入一道作业的代码（数据）。

1. 划分分区的方法



等分方式



差分方式

优点：算法实现简单，无外部碎片；

缺点：存在浪费（一次性全部分配一个分区），分区内有**内部碎片**。

4.3.2 固定分区分配

2. 内存分配

为了便于内存分配，通常将分区按大小排队，建立一张分区说明表。**表项**：分区起始地址、大小、状态（是否已分配）。

分区号	大小(K)	始址(K)	状态
1	12	20	已分配
2	32	32	已分配
3	64	64	未分配
4	128	128	未分配

图4-5(a) 分区说明表

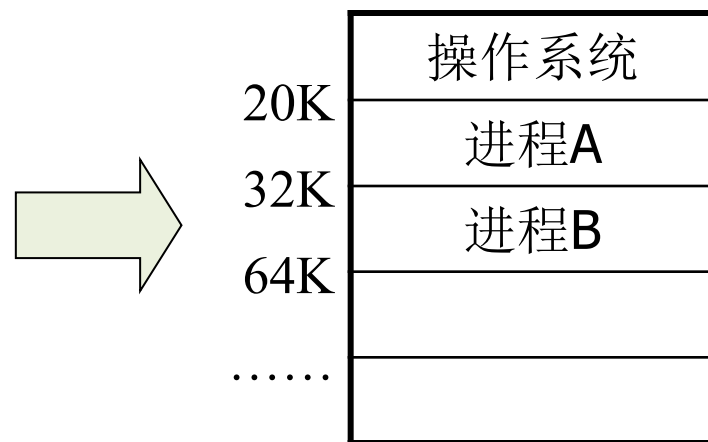


图4-5(b) 内存分配情况

分配：查分区说明表，找到一个足够大的空闲分区分配之；

回收：将回收分区对应的分区说明表状态改为“空闲”。



4.3.3 动态分区分配（可变分区分配）

- ❑ 思想：内存**事先不划分分区**，待作业需要分配内存时，再**按需分配**划分分区（分区的大小及个数不固定）。
- ❑ 算法实现：根据进程的实际需要，动态地为之分配内存空间。在实现时，建立相关数据结构，根据分配策略（或算法）设定算法程序。涉及到3个问题：**数据结构、分配算法、分配及回收操作**。

如何进行分区的分配与回收操作？

记录内存的使用情况

分配哪个空闲分区？



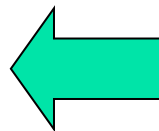
1. 分区分配中的数据结构

□ 可以有两种形式：空闲分区表、空闲分区链。

1) 空闲分区表

每个空闲分区占一个表目，表目包括：**分区号**、**分区始址**、**分区大小**等

分区号	分区始址	分区长度	状态
1	100KB	128KB	空闲
2	800KB	200KB	空闲
3	1300KB	700KB	空闲

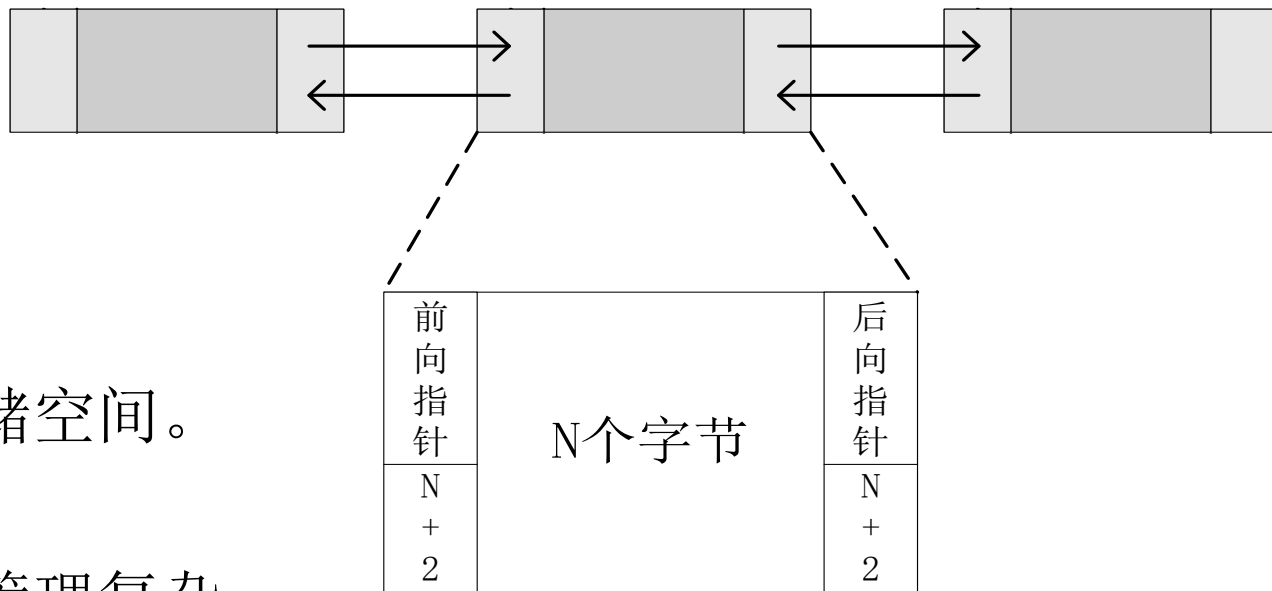




1. 分区分配中的数据结构

2) 空闲分区链

每个分区的起始部分设置前向指针和一些用于控制分区分配的信息；末尾部分设置后向指针。



优点:

自身不占用存储空间。

缺点:

比空闲分区表管理复杂。

优点：有效保留高址部分的大空闲分区；

缺点：低址部分不断被划分，会产生碎片；查找开销大。

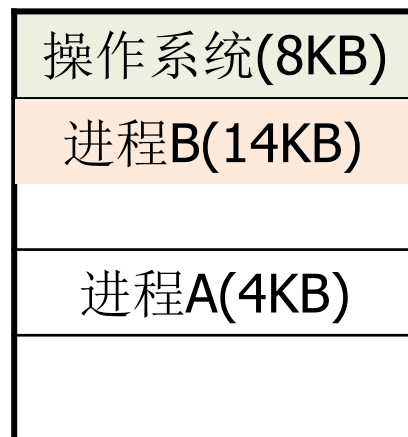
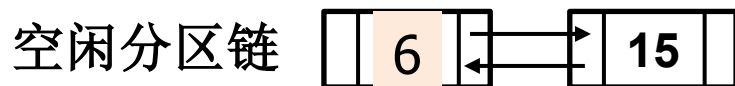
2. 分区分配算法

1) 基于顺序搜索的动态分区分配算法 (4.3.4)

□ 首次适应算法(First Fit)

空闲分区按**地址递增排序**。每次分配内存时从**低地址**开始，**按顺序查找**，直至找到一个大小能满足要求的空闲分区；然后按作业大小划出一块内存空间分配给请求者，余下的空闲分区仍留在链中。

空闲分区表				进程B(14KB)	
分区号	分区始址	分区长度	状态	20KB	
1	22 KB	6 KB	空闲		
2	32KB	15KB	空闲	15KB	



优点：减小查找开销；

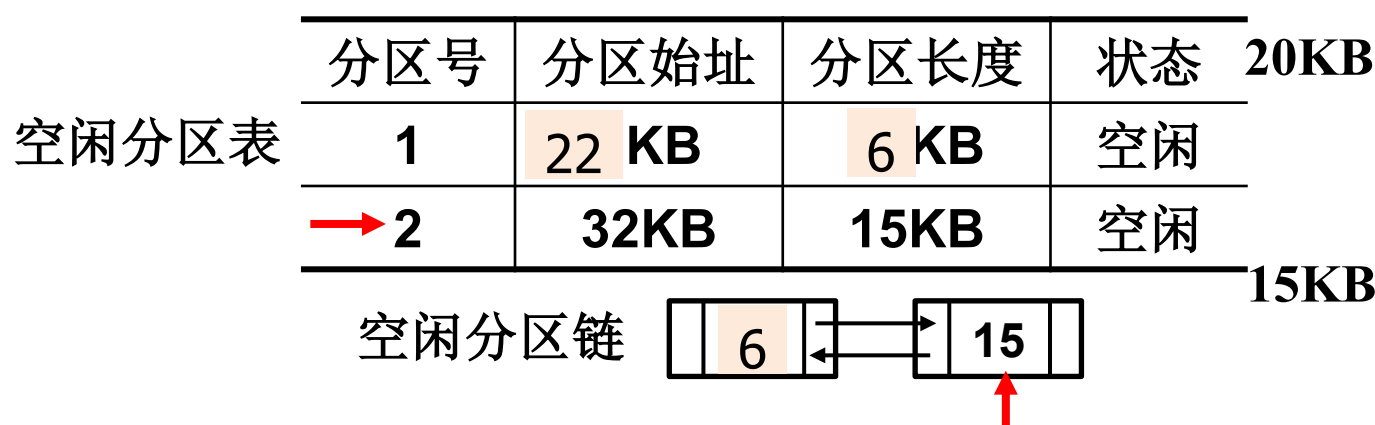
缺点：缺乏大的空闲分区。

2. 分区分配算法

1) 基于顺序搜索的动态分区分配算法 (4.3.4)

□ 循环首次适应算法(Next Fit)

由首次适应算法演变而成。分配内存时，从上次找到的空闲分区的**下一个空闲分区**开始查找，直至找到一个大小能满足要求的空闲分区，从中划出一块与请求大小相等的分区分配给作业。为实现该算法，应设置一**起始查找指针**。



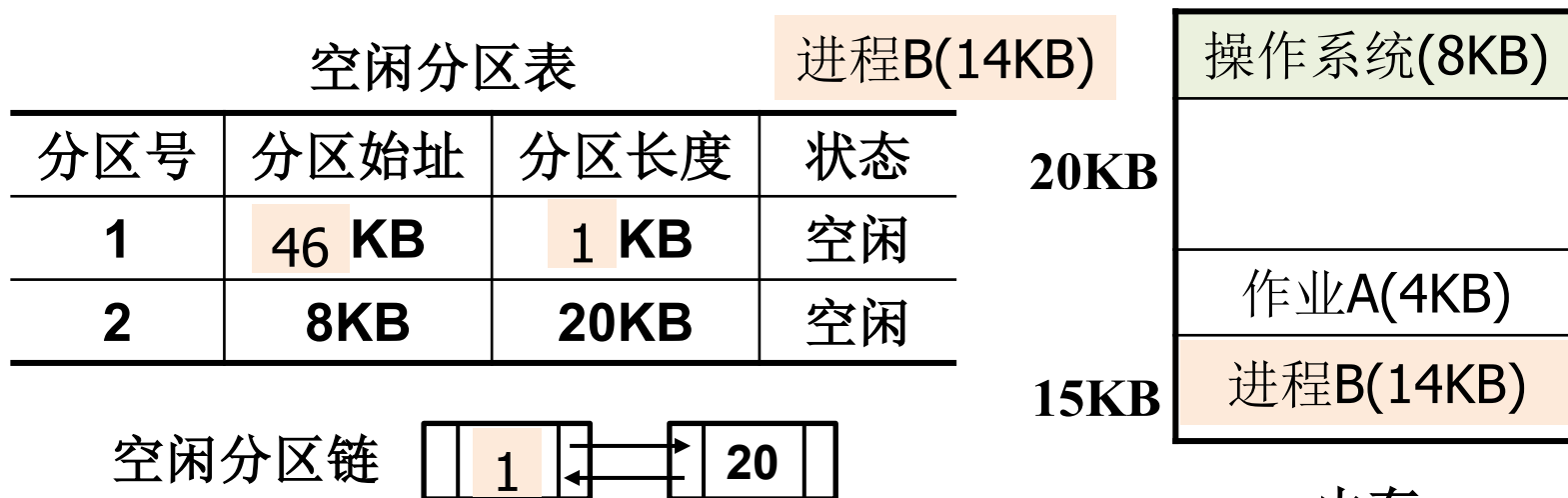


2. 分区分配算法

1) 基于顺序搜索的动态分区分配算法 (4.3.4)

□ 最佳适应算法(Best Fit)

空闲分区**按容量递增排序**。每次分配时，总是将能满足要求的**最小分区**分配给请求者。



内存

缺点：有很多难以利用的碎片。

优点：使产生的碎片尽可能小。

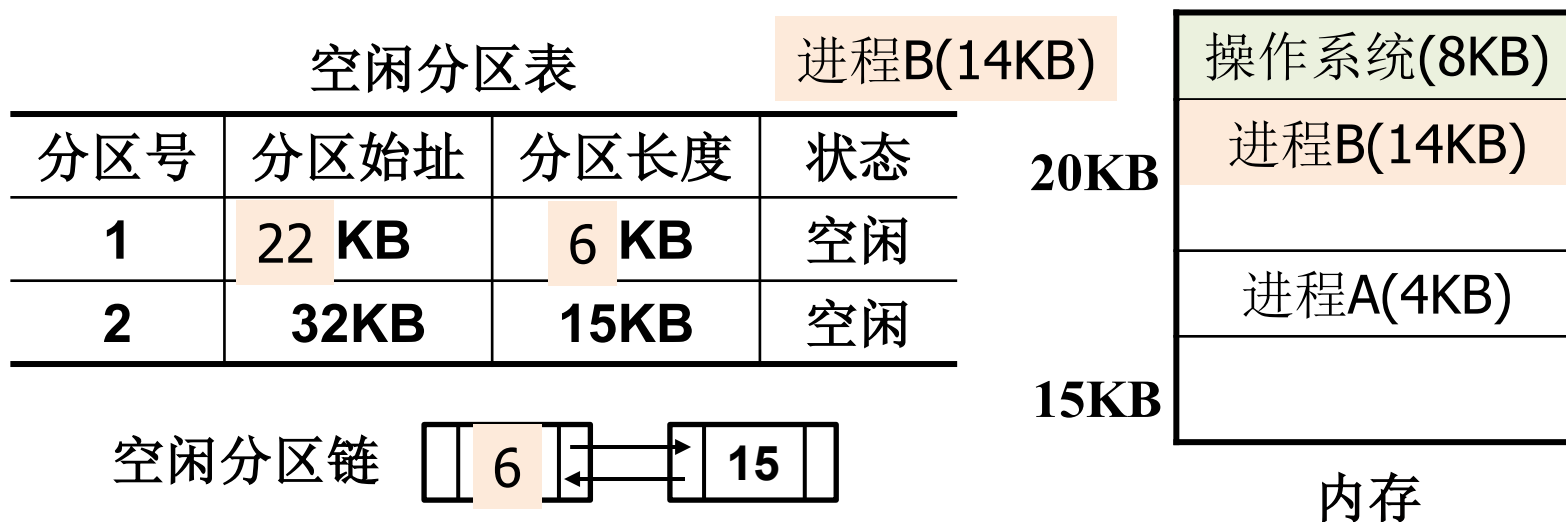
缺点：不利于大进程执行。

2. 分区分配算法

1) 基于顺序搜索的动态分区分配算法 (4.3.4)

□ 最坏适应算法(Worst Fit)

空闲分区**按容量递减排序**。每次分配时，总是将能满足要求的**最大分区**分配给请求者。





例1

某系统采用动态分区分配方式管理内存，内存空间为640K，**高端40K**用来存放操作系统。在内存分配时，系统**优先使用空闲区低端**的空间。对下列的请求序列：作业1申请130K、作业2申请60K、作业3申请100K、作业2释放60K、作业4申请200K、作业3释放100K、作业1释放130K、作业5申请140K、作业6申请60K、作业7申请50K、作业6释放60K。

请分别画图表示出使用**首次适应算法**、**最佳适应算法**进行内存分配和回收后内存的实际使用情况。

作业1申130K、作业2申60K、作业3申100K、作业2释60K、作业4申200K、作业3释100K、作业1释130K、作业5申140K、作业6申60K、作业7申50K、作业6释60K。

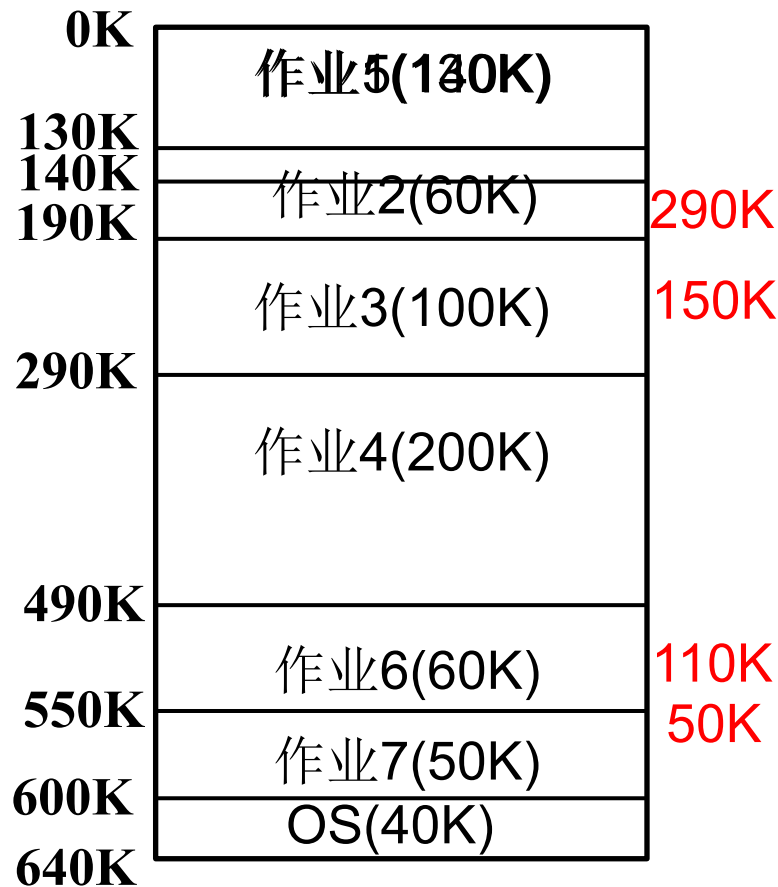


例1解答



首次适应算法

从低地址查找



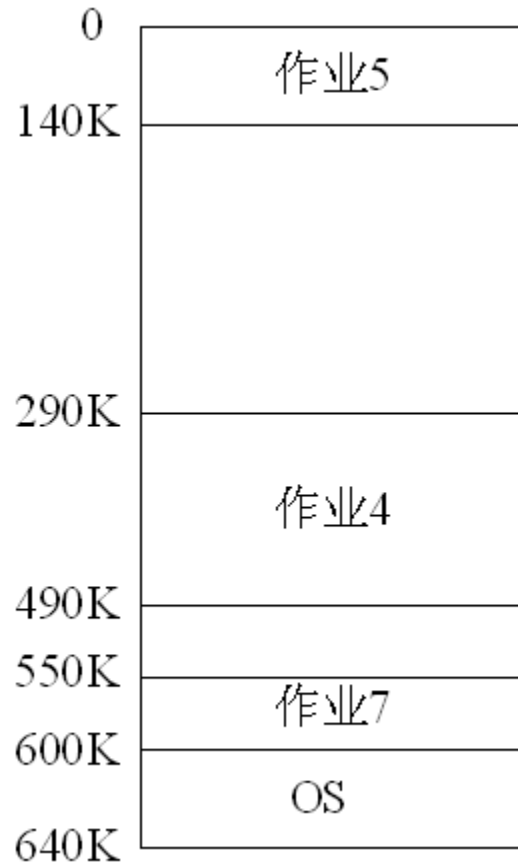
最佳适应算法 从小容量查找



例1解答



首次适应算法



最佳适应算法



例2

在可变分区管理系统中，假定主存中按地址顺序依次有5个空闲区，其大小依次为32KB、10KB、5KB、228KB、100KB，现有5个作业J1、J2、J3、J4、J5，它们各需主存1KB、10KB、108KB、28KB、115KB，则内存分配采用（ ）适应算法，可将5个作业全部装入内存？

- A. 首次 B. 循环首次（下次） C. 最佳 D. 最坏

优点：能保留大的分区，查找效率高。

缺点：分区回收时算法复杂，系统开销大。

2. 分区分配算法

2) 基于索引搜索的动态分区分配算法 (4.3.5)

□ 快速适应算法 (Quick Fit)

空闲分区按容量大小进行分类。对每一类具有相同容量的所有空闲分区，单独设立一个空闲分区链表。同时，在内存中设立一张管理索引表，其中的每一个索引表项对应了一种空闲分区类型，并记录了该类型空闲分区链表表头的指针。

算法实现：

- 第一步，根据进程长度，从索引表中寻找能容纳它的最小空闲分区链表；
- 第二步，从链表中取下第一块进行分配即可。



2. 分区分配算法

2) 基于索引搜索的动态分区分配算法 (4.3.5)

□ 伙伴系统 (Buddy System)

该算法规定，无论已分配分区或空闲分区，其大小均为 2^k (k 为整数， $1 \leq k \leq m$ ，最大可分配内存为 2^m)。对于每一类具有相同容量的所有空闲分区，单独设立一个空闲分区链表。当为进程分配一个长度为 n 的存储区域时，计算 i 值，使得 $2^{i-1} < n \leq 2^i$ 然后在大小为 2^i 的空闲分区链表中查找。若找到，把空闲分区分配给进程；若未找到，则在大小为 2^{i+1} 的空闲分区链表中查找。若找到，把该空闲分区分为相等的两个分区（称为**一对伙伴**），一个用于分配，另一个加入分区大小为 2^i 的空闲分区链表中。



2. 分区分配算法

2) 基于索引搜索的动态分区分配算法 (4.3.5)

□ 哈希算法 (Buddy System)

利用哈希快速查找的优点以及空闲分区在空闲分区表中的规律，建立哈希函数，构造以空闲分区大小为关键字的哈希表，该表每个表项记录一个对应的空闲分区链表表头指针。进行空闲分区分配时，根据空闲分区大小，通过哈希函数计算，得到在哈希表中的位置，从而得到空闲分区链表的指针。



3. 分区的分配和回收操作

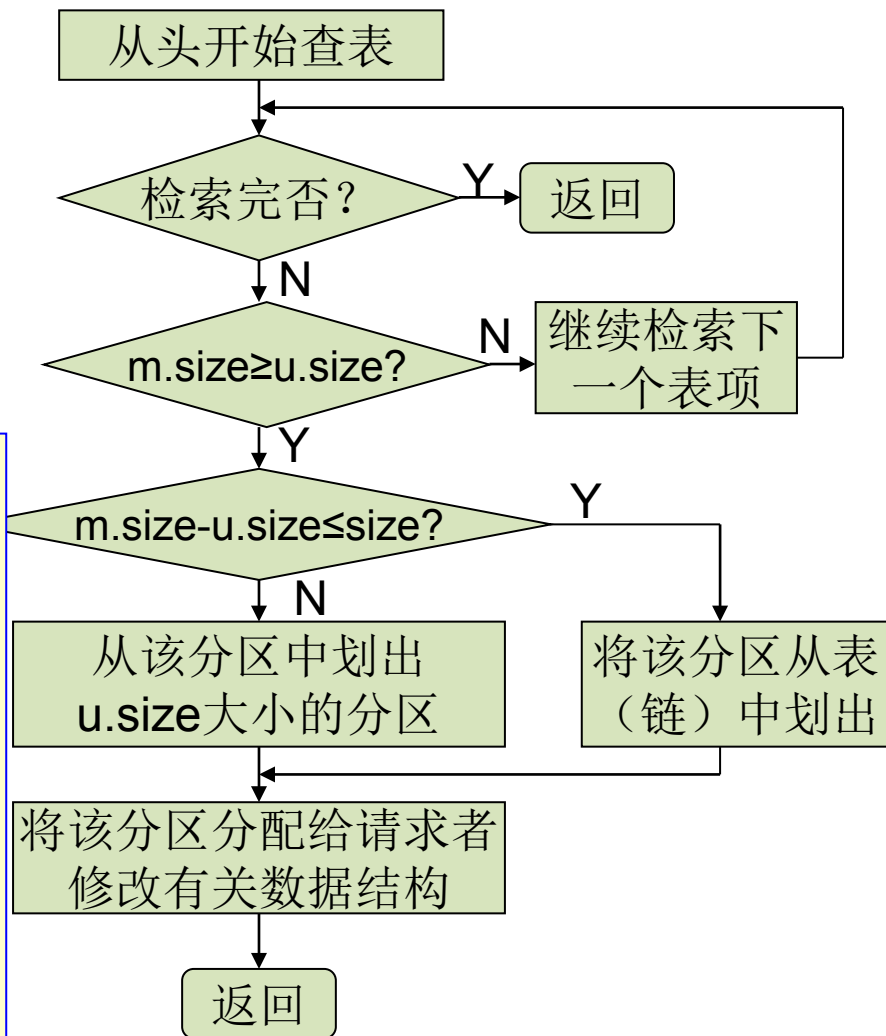
1) 分配内存

分配内存过程如图4-8所示：

- 请求分区大小 $u.size$ ；
- 分区大小为 $m.size$ ；
- 不再切割的剩余分区大小为 $size$ 。

当某空闲分区满足 $m.size \geq u.size$ 时，执行如下操作：

- ①当 $m.size - u.size \leq size$ 时，将整个分区分配给请求者；
- ②否则，按作业大小划出一块内存空间分配给请求者，余下的空闲分区仍留在空闲分区表（链）中。
- ③将分配区的首地址返回给调用者。





3. 分区的分配和回收操作

2) 回收内存

根据回收区首地址，在空闲分区表（链）找到插入点，此时可能出现4种情况之一：

- 回收区与插入点前一个空闲分区**F1**相邻接：将回收区与前一分区合并，不增加新表项，只需修改**F1**的大小为两者之和；

	操作系统
10K	进程A
22K	空闲区 F1
80K	回收区
100K	进程B
130K	

分区号	分区始址	分区长度	状态
1	22KB	78 KB	空闲



3. 分区的分配和回收操作

2) 回收内存

- 回收区与高地址空闲分区**F2**邻接：将回收分区与该空闲分区合并，回收区的首地址为新分区的首地址，大小为两者之和。

	操作系统
10K	进程A
22K	进程B
80K	回收区
100K	空闲区F2
130K	

分区号	分区始址	分区长度	状态
1	80 KB	50 KB	空闲



3. 分区的分配和回收操作

2) 回收内存

- 回收区与前后空闲分区**F1**和**F2**都邻接：将此3个分区合并，前邻接区**F1**的首地址为新分区的首址，大小为三者之和，取消**F2**表项。

	操作系统
10K	进程A
22K	
80K	空闲区F1
	回收区
100K	
130K	空闲区F2

分区号	分区始址	分区长度	状态
1	22KB	58KB	空闲
2	100KB	30KB	空闲

分区号	分区始址	分区长度	状态
1	22KB	108KB	空闲



3. 分区的分配和回收操作

2) 回收内存

- 回收区与任何空闲区都不邻接：在插入点建立一个新表项，填写回收区的首地址和大小。插入到空闲区表的适当位置（后移插入点后的各个表项）。

	操作系统
10K	进程A
22K	进程B
80K	空闲区
100K	进程C
130K	

分区号	分区始址	分区长度	状态
1	80KB	20KB	空闲



4.3.6 可重定位分区分配

1. 紧凑

- 目的：为了解决内存“碎片”问题。 **外部碎片**
- 方法：移动内存中作业的位置，把原来多个分散的小分区拼接成一个大分区，称为“拼接”或“紧凑”。每次紧凑后，都必须对移动的程序或数据进行**动态重定位**。



(a) 紧凑前

(b) 紧凑后



4.3.6 可重定位分区分配

2. 动态重定位

- **动态重定位的定义：**地址变换过程是在程序执行期间，随着对每条指令或数据的访问自动进行的。
- 为使地址转换不会影响到指令执行速度，必须有**硬件地址变换机构**——增设一个重定位寄存器，用于存放程序（数据）在内存中的起始地址。

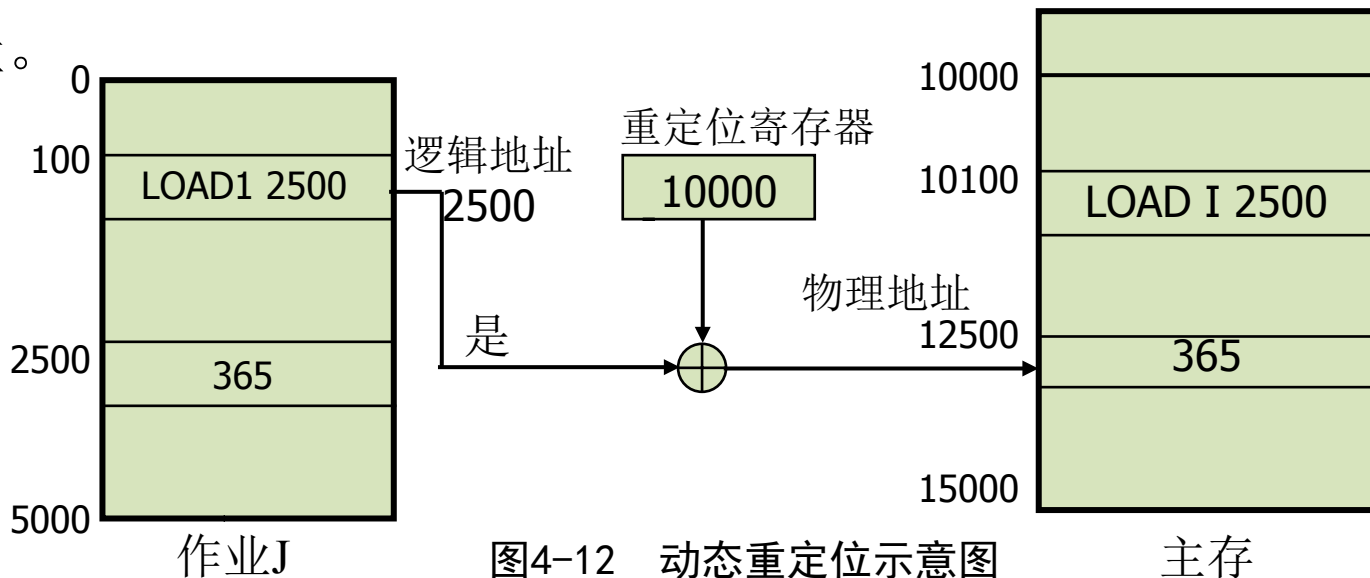


图4-12 动态重定位示意图

主存



4.3.6 可重定位分区分配

3. 动态重定位分区分配算法

□ **算法思想：**在可变分区分配算法的基础上，采用动态重定位方式装入程序（数据）。当无足够大的分区供分配时，若总空闲存储容量够用，则将各分区中的内容向内存一端移动（紧凑），使另一端形成一个大的空闲分区，然后再分配。

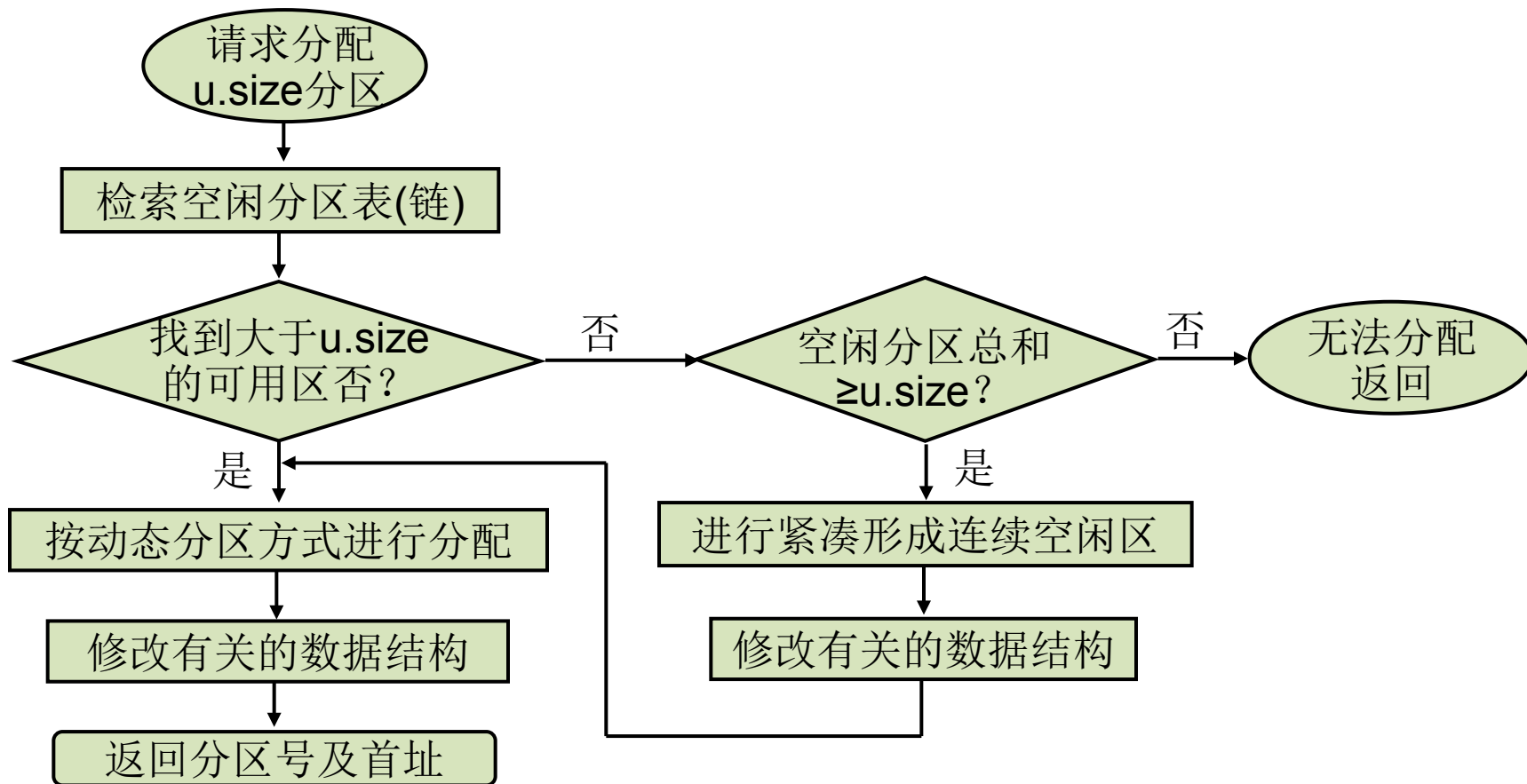
与动态分区分配算法基本相同，只是增加了紧凑功能。

当找不到满足请求的空闲区，但空闲区总和大于等于请求数时，进行紧凑操作。



4.3.6 可重定位分区分配

3. 动态重定位分区分配算法



- 1、在可变式分区存储管理中的拼接技术可以（**A**）。
A. 集中空闲区 B. 增加主存容量
C. 缩短访问周期 D. 加速地址转换
- 2、分区管理中采用“最佳适应”分配算法时，宜把空闲区按（**A**）次序登记在空闲区表中。
A. 长度递增 B. 长度递减
C. 地址递增 D. 地址递减
- 3、在固定分区分配中，每个分区的大小是（**C**）。
A. 相同
B. 随作业长度变化
C. 可以不同但预先固定
D. 可以不同但根据作业长度固定



4、把作业地址空间中使用的逻辑地址变成内存中物理地址的过程称为地址变换 / 重定位。

5、首次适应算法的空闲区是（**A**）。

- A. 按地址递增顺序连在一起
- B. 始端指针表指向最大空闲区
- C. 按大小递增顺序连在一起
- D. 寻找从最大空闲区开始





4.4 对换

- 4.4.1 多道程序环境下的对换技术
- 4.4.2 对换空间的管理
- 4.4.3 进程的换出与换入



4.4.1 多道程序环境下的对换技术

1. 对换的引入

- 阻塞进程占用大量内存
- 外存中的等待作业因无内存不能调入运行

资源浪费

对换：把内存中暂时不能运行的进程或进程所需要的程序和数据，调出到外存上，以便腾出足够的内存空间，再把已具有运行条件的进程或进程所需要的程序和数据，调入内存。



4.4.1 多道程序环境下的对换技术

2. 对换的类型

整体对换 (进程对换)

- 对换以整个进程为单位
- 广泛应用于分时系统中

中级调度

部分对换

- 对换以“页”为单位——页面对换
- 对换以“段”为单位——分段对换

虚存

本节介绍进程对换，部分对换将在虚拟存储器中介绍。

要实现进程对换，需实现三方面功能：

- 对换空间的管理
- 进程的换出
- 进程的换入



4.4.2 对换空间的管理

把外存分为：**文件区和对换区**

文件区

- 存放文件
- 管理主要目标：提高文件存储空间利用率
- 为此，采用离散分配方式

对换区

- 存放换出的进程
- 管理主要目标：提高进程换入和换出的速度
- 为此，采用连续分配方式

数据结构：对换区的空闲盘块管理的数据结构与动态分区分配方式采用的数据结构相似——**空闲区表或空闲区链**。

空闲分区表中应包含两个表项：对换区的首址及其大小，它们的单位是盘块号和盘块数。

分配算法：对换区的分配与回收，与动态分区分配雷同，其分配算法可以是首次适应、循环首次适应或最佳适应分配算法。



4.4.3 进程的换出与换入

进程的 换出

当一进程由于创建子进程而需要更多内存空间，但无足够的内存空间等情况发生时，系统应将某个进程换出。

换出过程：

- 选择处于**阻塞状态**且**优先级别最低**的进程作为换出进程；
- 启动磁盘，将该进程的程序和数据传送到磁盘的**对换区**上；
- 若传送过程未出现错误，回收该进程所占用的内存空间，并对该进程的**PCB**做相应的修改。

进程的 换入

系统应定时查看所有进程的状态，从中找出“**就绪**”状态但已换出的进程，将其中**换出时间最久**的进程换入，直至已无可换入的进程或无可换出的进程为止。



4.5 分页存储管理方式

4.3

连续内存分配方式
(分区管理方式)



离散内存分配方式

进程占用内存
离散空间
需要解决地址
转换问题

单一连续分配
固定分区分配
动态分区分配
可重定位分区分配

缺点:

1. 连续分配方式会形成“碎片”，内存利用率较低；
2. “紧凑”须付出很大开销。

- 分页（页式）存储管理方式 4.5
- 分段（段式）存储管理方式 4.6
 - 段页式存储管理方式
- 虚拟存储器
 - 请求分页存储管理方式
 - 请求分段存储管理方式



4.5 分页存储管理方式 最大的优点是内存利用率高

□ 4.5.1 分页管理的基本方法

□ 4.5.2 地址变换机构

~~□ 4.5.3 访问内存的有效时间~~

□ 4.5.4 两级和多级页表

~~□ 4.5.5 反置页表~~



4.5.1 分页管理的基本方法

分页由系统完成

1. 页面和物理块

- 物理块：将物理内存空间分成大小相等的若干存储块，称为块或页框(frame)。从0开始编号。
- 页面：逻辑地址空间按物理块大小分成若干片，称为页面或页(page)。页号也从0开始。

若进程最后一个页不满，将产生页内碎片。

- 页面大小：应是2的幂。通常为512B~8KB。

不宜过小，也不宜过大

商用计算机的页面大小在512B~64KB之间，以往的典型值为1KB，而现在更常见的页面大小为4KB和8KB。参见《现代操作系统》。



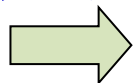
4.5.1 分页管理的基本方法

块号

0	
1	
2	
3	
4	
5	
6	
7	
8	
	⋮

内存

进程A
共2页

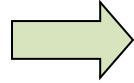


块号

0	A.0
1	A.1
2	
3	
4	
5	
6	
7	
8	
	⋮

内存

进程B
共3页

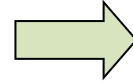


块号

0	A.0
1	A.1
2	B.0
3	B.1
4	B.2
5	
6	
7	
8	
	⋮

内存

进程A
结束





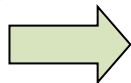
4.5.1 分页管理的基本方法

块号

0	
1	
2	B.0
3	B.1
4	B.2
5	
6	
7	
8	
	⋮

内存

进程C
共3页



块号

0	C.0
1	C.1
2	B.0
3	B.1
4	B.2
5	C.2
6	
7	
8	
	⋮

内存

假设图中逻辑地址为32位，每页大小为4KB，地址空间最多 2^{20} （1M）个页。

4.5.1 分页管理的基本方法

2. 地址结构

□ 分页系统中的进程逻辑地址结构如下：



□ 分页系统中的内存地址（物理地址）结构如下：





4.5.1 分页管理的基本方法

2. 地址结构

对于特定的机器，其地址结构是一定的。若逻辑地址为A，页面大小为L，则页号P和页内地址W可按下式求得：

$$P = \text{INT}\left[\frac{A}{L}\right], W = A \text{ MOD } L$$

例：页面大小为**4KB**，逻辑地址为4170及5F86H，分别求它们的页号和页内地址。

【A=4170】

页号：1；页内地址：74

【B=5F86H】

页号：5；页内地址：F86H或3974

思考题：如何求每个页表项的大小？

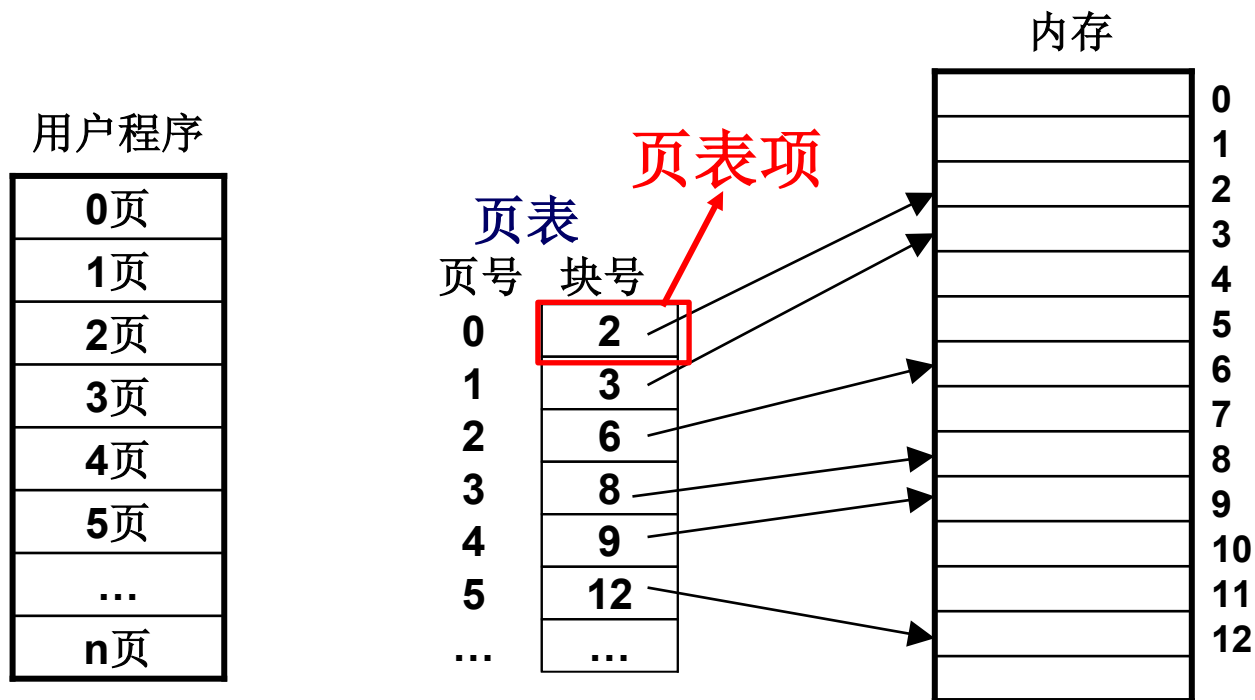
页表项连续存放，因此页号隐含，不占内存。

每个页表项大小相等，
取决于物理块号位数。

4.5.1 分页管理的基本方法

3. 页表

系统为每个进程建立一张页表，记录了相应页在内存中对应的物理块号，实现从页号到物理块号的地址映射。





4.5.1 分页管理的基本方法

3. 页表（Page Table）

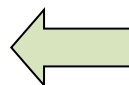
系统为**每个进程建立一张页表**，记录了**相应页在内存中对应的物理块号**，实现从页号到物理块号的地址映射。

进程B页表

页号	块号
0	2
1	3
2	4

进程C页表

页号	块号
0	0
1	1
2	5



块号

0	C.0
1	C.1
2	B.0
3	B.1
4	B.2
5	C.2
6	
7	
8	
	⋮

系统会为内存建立一个**空闲页框表**（free-frame list）

内存空闲页框表

块号

6
7
8
⋮

内存

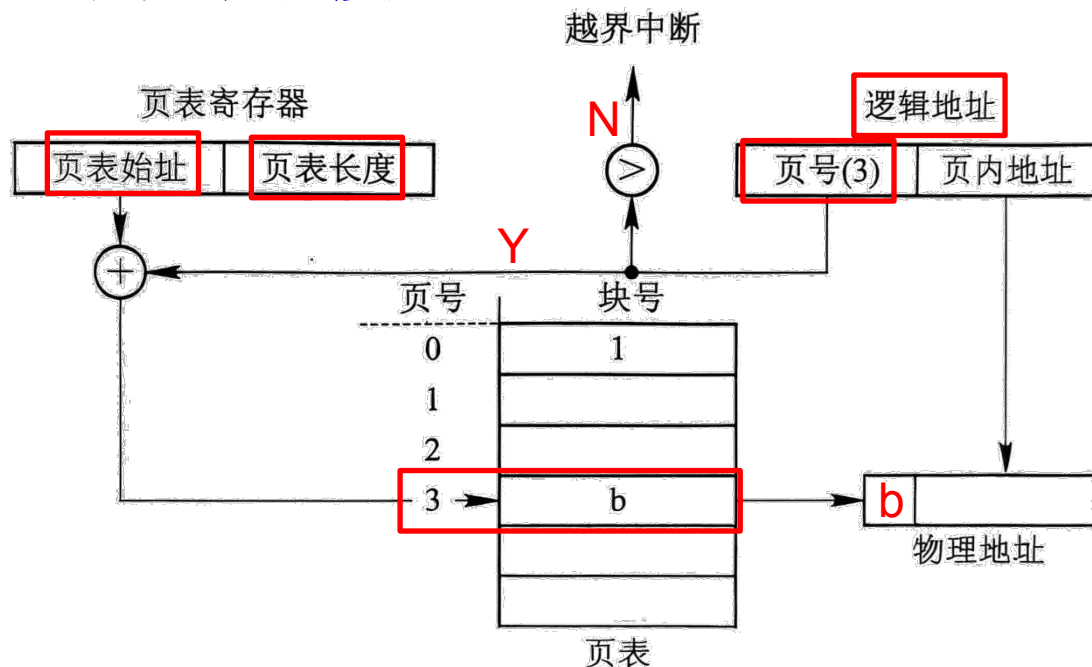


4.5.2 地址变换机构

硬件MMU（存储管理单元），实现逻辑-物理地址转换

1. 基本的地址转换机构

页表大多驻留在内存中，设置一个页表寄存器存放页表在内存中的始址和页表长度。



页号3对应的页表项的物理地址始址 = 页表始址 + 3 × 页表项长度



4.5.2 地址变换机构

1. 基本的地址转换机构 工作原理

设逻辑地址2056，假定一页大小为1K（1024B），其中0号页放到了3号块中，1号页放到了5号块，2号页放到了8号块，求此逻辑地址对应的物理地址。

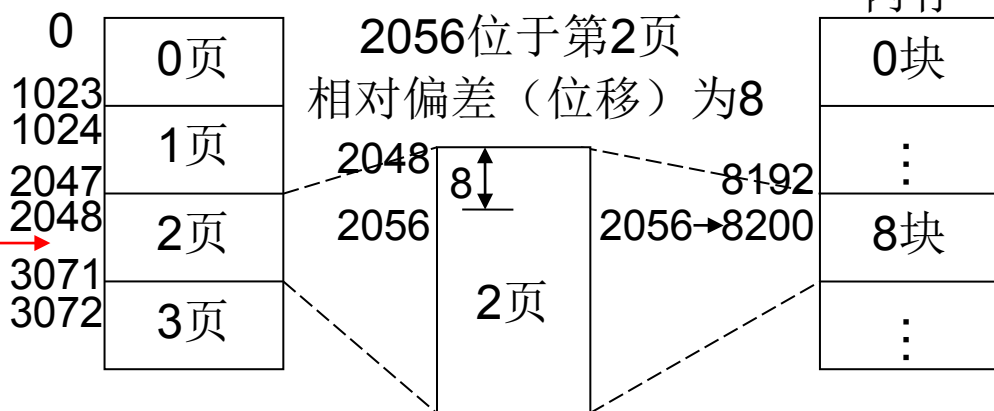
物理地址？
内存

方法：

- 逻辑地址/页大小 取整
 $\frac{2056}{1024} = 2$
 $\rightarrow$ 页号 $\rightarrow$ 块号

- 逻辑地址/页大小 取余
 $2056 \div 1024 = 2 \text{ 余 } 8$
 $\rightarrow$ 页内位移量 8

- 块号*页大小+页内位移量 $\rightarrow$ 物理地址
 $8 \times 1024 + 8 = 8200$

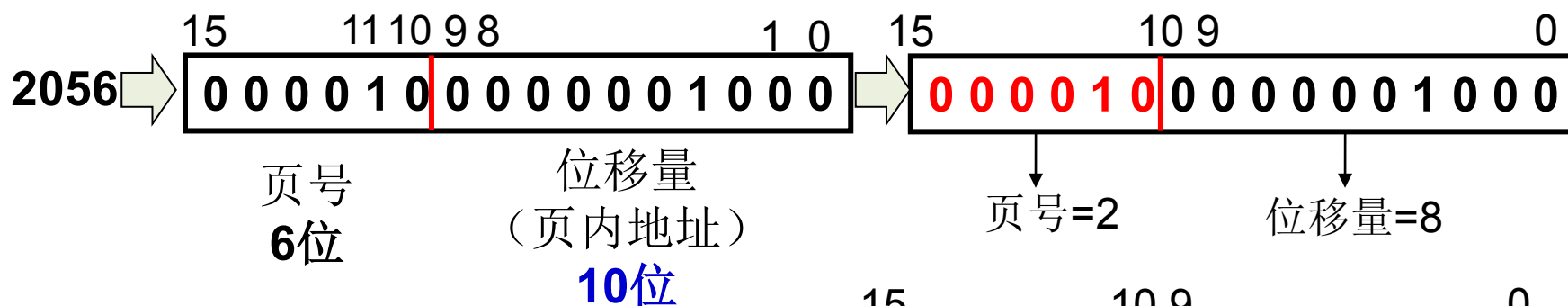




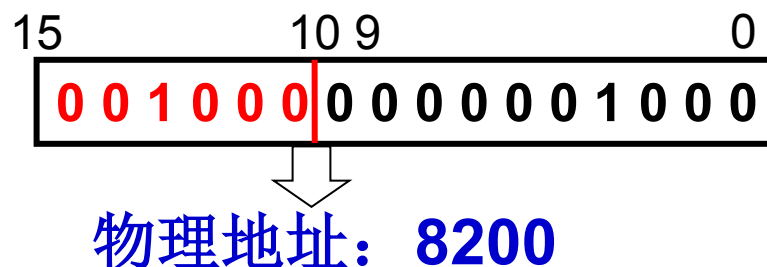
4.5.2 地址变换机构

* 转换过程分析

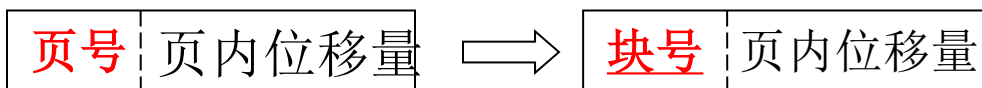
假设逻辑地址机内表示（以16位为例） 页大小为 $1\text{KB}=2^{10}\text{B}$



将页号转换为设定的块号，则有：



无需计算，用块号代替高位的页号，就可得对应的物理地址





例题

在采用页式存储管理的系统中，某作业的逻辑地址空间为4页，每页4096字节，且已知该作业的页表如下表。试求出逻辑地址14688所对应的物理地址。

解：

$$\text{页号 } P = \text{INT} \left[\frac{14688}{4096} \right] = 3$$

$$\begin{aligned} \text{页内位移量 } W &= 14688 \text{ MOD } 4096 \\ &= 2400(960\text{H}) \end{aligned}$$

$$\text{物理地址} = 9 \times 4096 + 2400 = 39264(9960\text{H})$$

页 表

页 号	内存快号
0	2
1	4
2	7
3	9

1. 设有一页式存储管理系统，向用户提供的逻辑地址空间最大为16页，每页2048B，内存总共有8个存储块。试问逻辑地址至少应为多少位？内存空间有多大？

15

16KB

2. 在一分页存储管理系统中，逻辑地址长度为24位，页面大小为4096B，现有一逻辑地址为2FA6H，且第0、1、2页依次存放在物理块10、12、14中，问相应的物理地址为多少？

逻辑地址: **2FA6**

物理地址: **EFA6**





4.5.2 地址变换机构

2. 具有快表的地址转换机构

- 分页系统中，每存取一个数据，要访问2次内存（找页表，找数据），计算机处理速度降低近1/2。
- 为**提高地址转换速度**，增设一个具有并行查寻能力的特殊高速缓冲寄存器，称为**联想寄存器**，或称**快表（Translation Lookaside Buffer, TLB）**。
- 快表（TLB）中存放的是**当前访问的那些页表项**。其工作原理是程序访问的局部性原理。

加快页表检索速度



2. 具有快表的地址转换机构

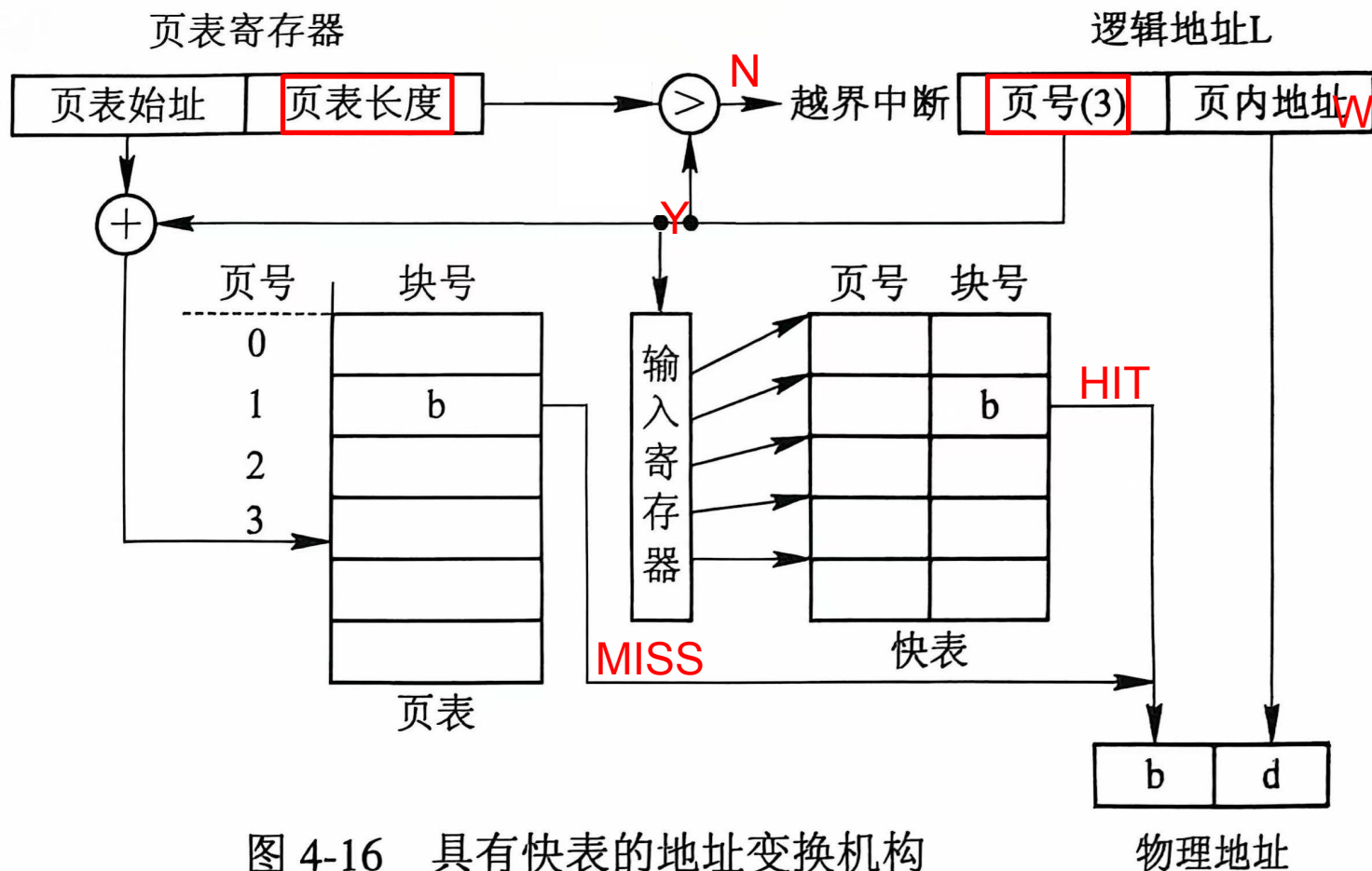
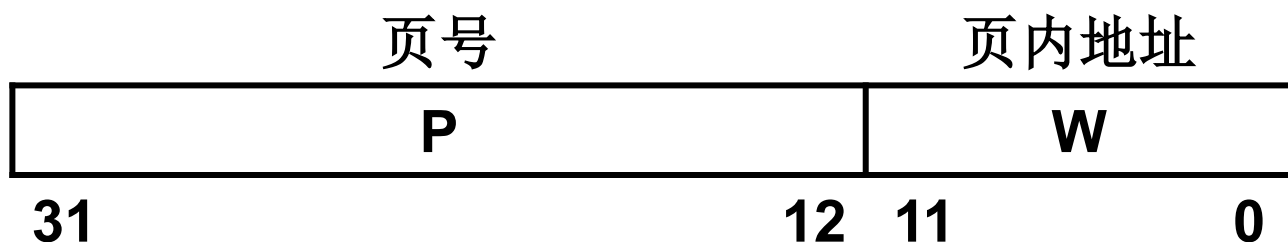


图 4-16 具有快表的地址变换机构



4.5.4 两级和多级页表

假设物理地址**44**位，某进程逻辑地址空间为**32**位，页面大小为**4KB**（**12**位）。逻辑地址结构如下：



页表中有 2^{20} 个页表项；每个页表项大小为： $44 - 12 = 32\text{b} = 4\text{B}$

页表大小为： $4\text{B} \times 2^{20} = 4\text{MB}$

该进程页表太大，难于找到一块连续的大内存空间 **多级页表**

由局部性原理，一段时间内可能只需要访问某几个特定的页面。
只将当前需要的部分页表项调入内存，其余的驻留在磁盘上，
需要时再调入。 **虚拟存储**



4.5.4 两级和多级页表

1. 两级页表

采用**离散方式**来解决难于找到一块连续的大内存空间存储页表的问题，解决的办法是：

- (1) 将页表分页（**组**）；
- (2) 将各个子页表（**小组**）离散地存放在内存不同物理块中；
- (3) 同时，再建立一张页表记录各个子页表（**小组**）在内存中的存放位置。此表称为**外层页表**，或**页目录表**，或**顶层页表**，或**一级页表**。

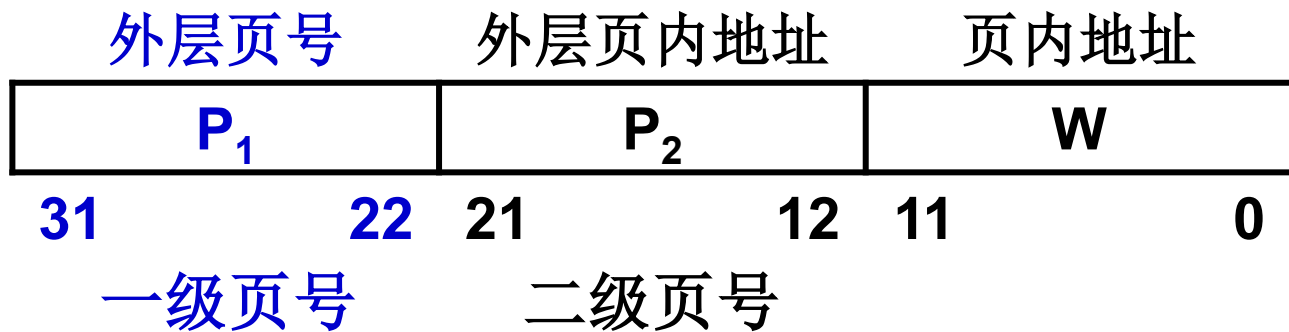
组号	组内地址	
外层页号 P_1	外层页内地址 P_2	页内地址 W
一级页号	二级页号	



4.5.4 两级和多级页表

1. 两级页表

【例】假设物理地址44位，逻辑地址空间为32位，页面大小为4KB（12位）。采用两级页表结构，再对页表分页，使每个子页表中包含 2^{10} （1024）个页表项，最多分成 2^{10} （1024）个子页表，即外部页表页号 P_1 为10位，外部页表的页内地址 P_2 也是10位。逻辑地址结构如下：





外层页号	外层页内地址	页内地址
10位	10位	12位



两级页表结构

页(组)号 块号

0	1011
1	1078
2	
...	...
1023	1742

外部页表
一级页表
(页表目录)

二级页表

0# 页表

0	1
1	4
2	6
...	...
1023	

1# 页表

0	114
1	115
...	...
1023	

1023# 页表

0	1468
1	
2	
...	...
1023	

一个进程最多有
1024个二级页表

每个进程一个页表目录，
每个页表目录中索引项最
多1024个

一个二级页
表映射4KB
内存空间，
正好占一块

0	
1	
2	
3	
4	
5	
6	
7	
...	...
114	
115	
...	...
1468	
...	...

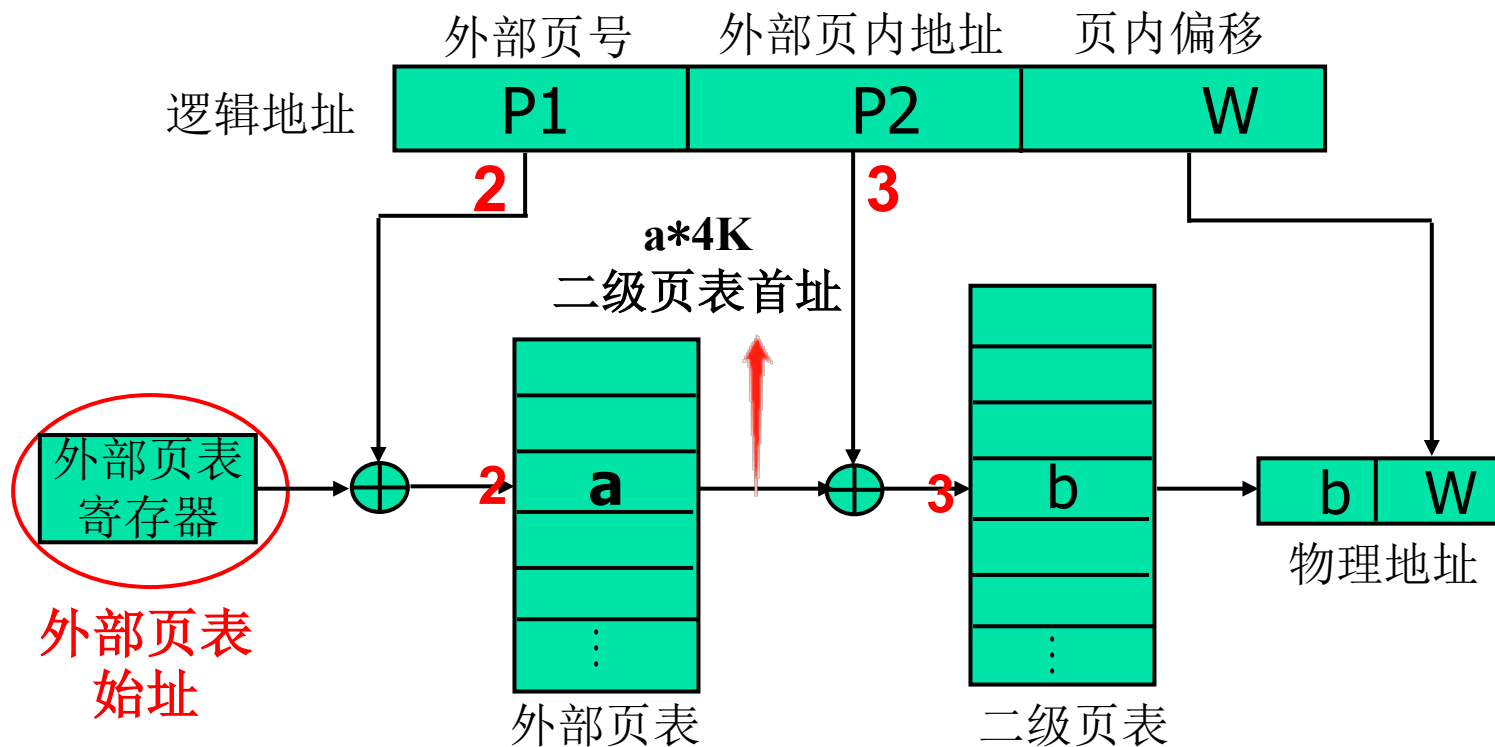
内存空间

【说明】本页注释是对上页的逻辑地址结构而言的。

4.5.4 两级和多级页表

1. 两级页表

设置一个**外层页表寄存器**，用于存放外层页表的始址。两级页表地址变换机构如下图所示（假设页面大小为4KB）。



1、在一个页式存储管理系统中，页表内容如下所示：

页号	块号
0	2
1	3
2	8

若页的大小为4K，则地址转换机构将逻辑地址8644转换成的物理地址为A。

A. 33220 B. 8644 C. 4548 D. 2500

2、要保证一个程序在主存中被改变了存放位置后仍能正确执行，则对主存空间 应采用（ A ）技术。

A. 动态重定位

B. 静态重定位

C. 动态分配

D. 静态分配



3、在可变分区存储管理中，某作业完成后要收回其主存空间，该空间可能要与相邻空闲区合并。在修改未分配区表时，使空闲区个数不变且空闲区始址不变的情况是（ **D** ）空闲区。

A. 无上邻也无下邻

B. 无上邻但有下邻

C. 有上邻也有下邻

D. 有上邻但无下邻

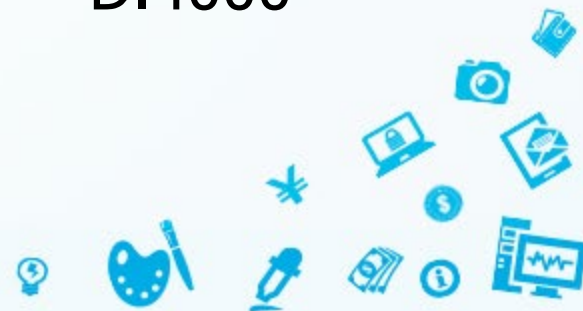
4、设基址寄存器内容为1000，在采用动态重定位的系统中，当执行指令“LOAD A,3000”时，操作数的实际地址是（ **D** ）。

A.1000

B. 2000

C.3000

D.4000



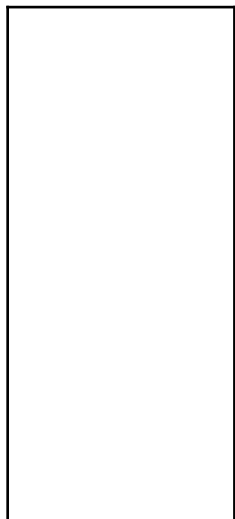


内容回顾



进程

内存用户区



连续分配：进程在内存占连续空间。

优：快

缺：碎片

进程

内存用户区



分页分配：进程按照块大小分页。可占内存不连续的块。

因此需要页表建立页号和块号对应关系，也存在内存中。



块0

块1

块2

块3

块4

块5

块6

块7

设页面大小为 L ，页表始址 F ，页表长度 M ，则逻辑地址 A 到物理地址 E 的变换过程如下（假设 L 、 A 、 E 都是十进制数）：

1. 根据逻辑地址计算：

$$\text{页号 } P = \text{INT} \left[\frac{A}{L} \right];$$

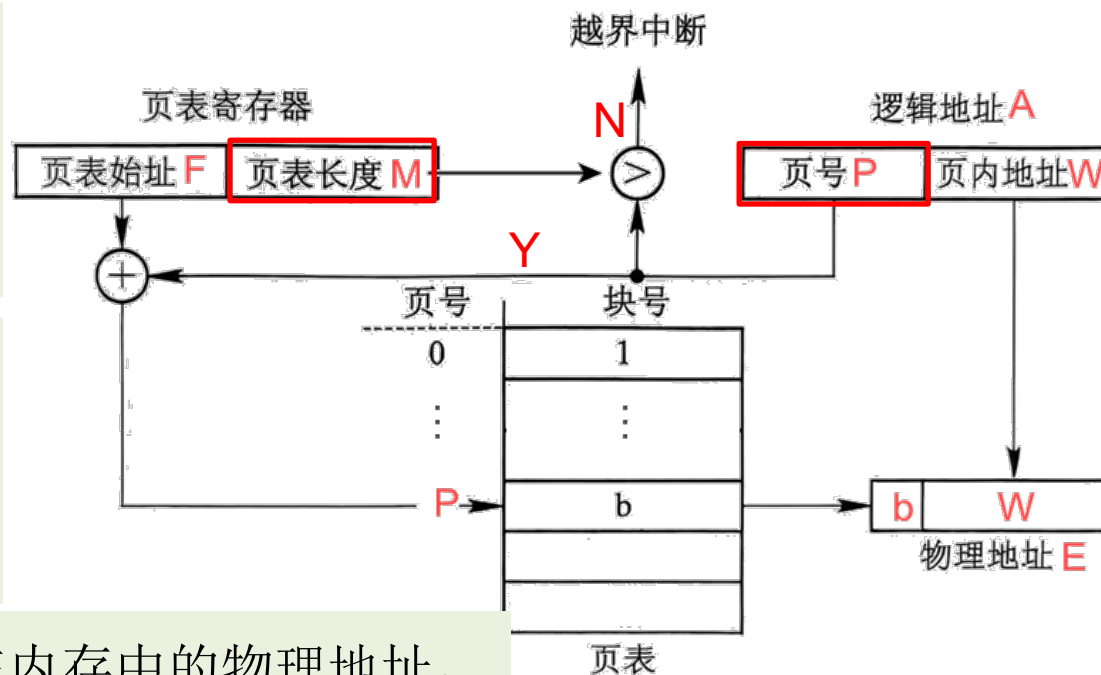
$$\text{页内偏移量 } W = A \bmod L;$$

2. 判断页号是否越界：若**页号** $\geq$ **页表长度** M ，则产生**越界**，否则继续执行；

3. 根据页表始址 F 表示的页表在内存中的物理地址，以及页号 P 查询**对应的页表项**，确定页面存放的物理块号 b 。

页号 P 对应的页表项的物理地址始址 = $F + P \times \text{页表项长度}$

4. 计算逻辑地址 A 对应物理地址 $E = b \times L + W$ ，用 E 访问主存，获取数据。





4.6 分段存储管理方式

- 4.6.1 分段存储管理方式的引入
- 4.6.2 分段系统的基本原理
- 4.6.3 信息共享
- 4.6.4 段页式存储管理方式



4.6.1 分段存储管理方式的引入

为了满足用户和程序员的下述需要：**作业分段由程序员完成**

方便
编程

通常用户把自己的程序按逻辑关系分为若干个段，每段都从0开始编址，并有自己的名字和长度。因此，希望要访问的逻辑地址是由段名(段号)和段内偏移量(段内地址)决定的。

信息
共享

在实现对程序和数据的共享时，是以信息的逻辑单位为基础的，比如，共享某个例程和函数。分页系统中的“页”只是存放信息的物理单位（块），并无完整的意义，不便于实现共享；而段却是信息的逻辑单位。

信息
保护

信息保护同样是对信息的逻辑单位进行保护，因此，分段管理能更有效地实现信息保护功能。

动态
增长

在实际应用中，往往有些段，特别是数据段，在使用过程中会不断增长，而事先又无法确切地知道数据段会增长到多大。前述的其它几种存储管理方式，都难以应付这种动态增长的情况，而分段存储管理方式却能较好地解决这一问题。

动态
连接

动态链接是指在作业运行之前，并不把几个目标程序链接起来。要运行时，先将主程序所对应的目标程序装入内存并启动运行，当运行过程中需要调用某段时，才将该段（目标程序）调入内存并进行链接。可见，动态链接也要求以段作为管理的单位。



4.6.2 分段系统的基本原理

1. 分段

- 程序员将作业地址空间划分成若干段，每段定义了一组**逻辑信息**。如：主程序段MAIN、子程序段X、数据段D、堆栈段S等。

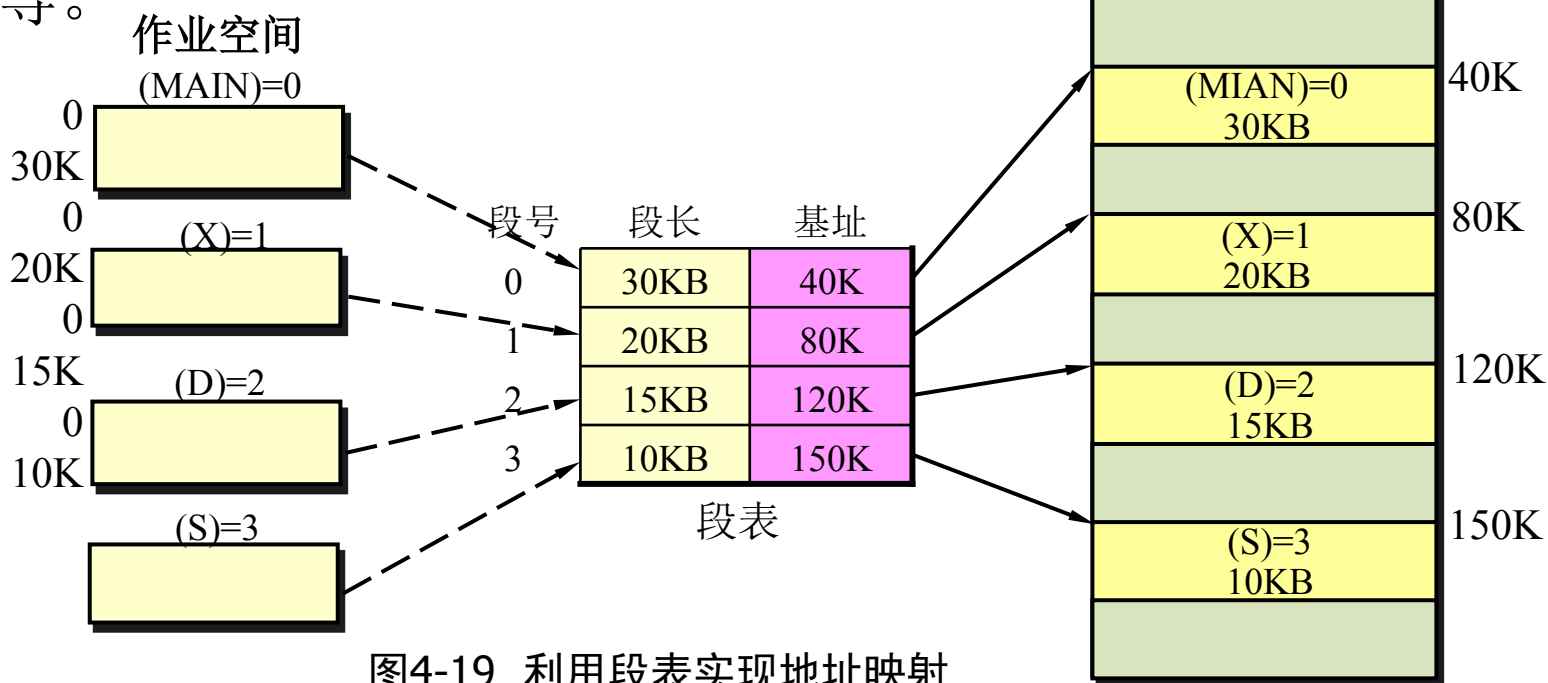


图4-19 利用段表实现地址映射

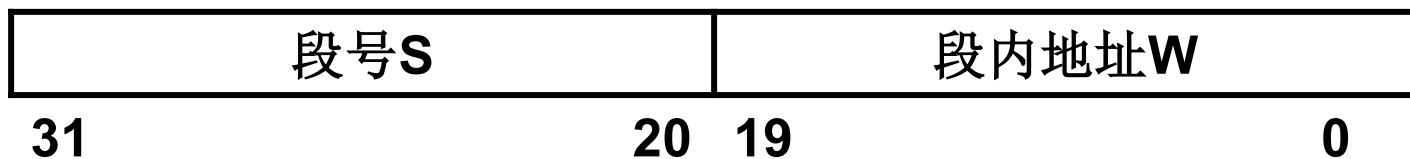


4.6.2 分段系统的基本原理

1. 分段

- 每个段都有名字。为实现简单，常用**段号代替段名**（段号从0开始）；
- 每个段内都从0开始编址，**每个段占一个连续内存空间**。多个段可不连续。由于分多个段，所以地址是二维的，亦即**逻辑地址由段号S和段内地址W**组成。具体结构举例如下：

段的大小不等，但为了方便判断是否越界，定义最大段长



该地址结构中允许作业最多有 $4K(2^{12})$ 个段，每段最大长度为 $1MB(2^{20})$ 。



4.6.2 分段系统的基本原理

2. 段表 段号 S 对应段表项的物理地址始址 = 段表始址 + $S \times$ 段表项长度

- 段表结构：段长，基址（段在内存中的始址）。
- 段表用于记载进程的各个段到物理内存中分区的映射情况。

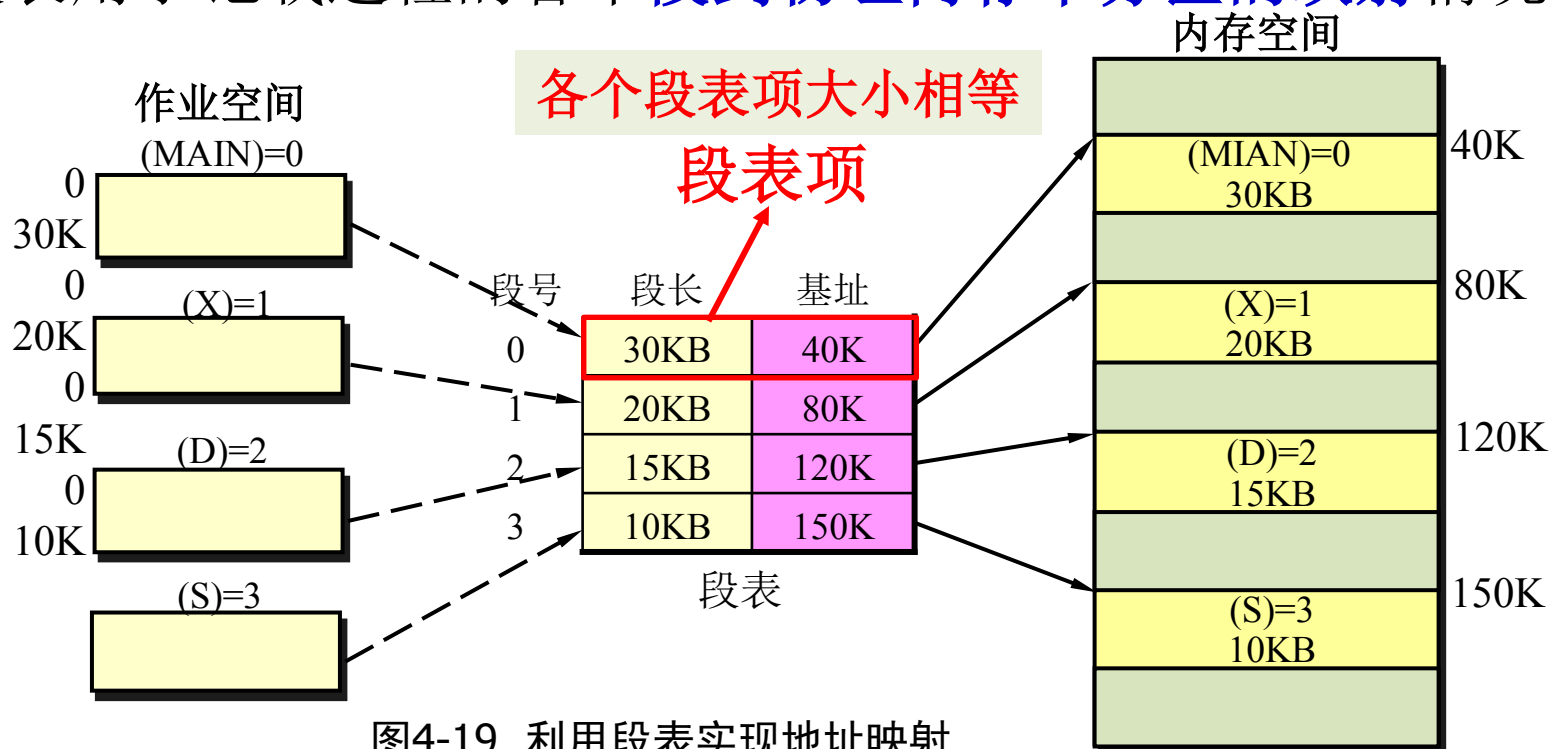


图4-19 利用段表实现地址映射

当段表存放在主存时，每访问一个数据，要访问两次内存，速度减慢1/2。但若将段表存于联想存储器中时，比没有地址变换的常规存储器存取速度仅减慢10%~15%。

4.6.2 分段系统的基本原理

3. 地址变换机构

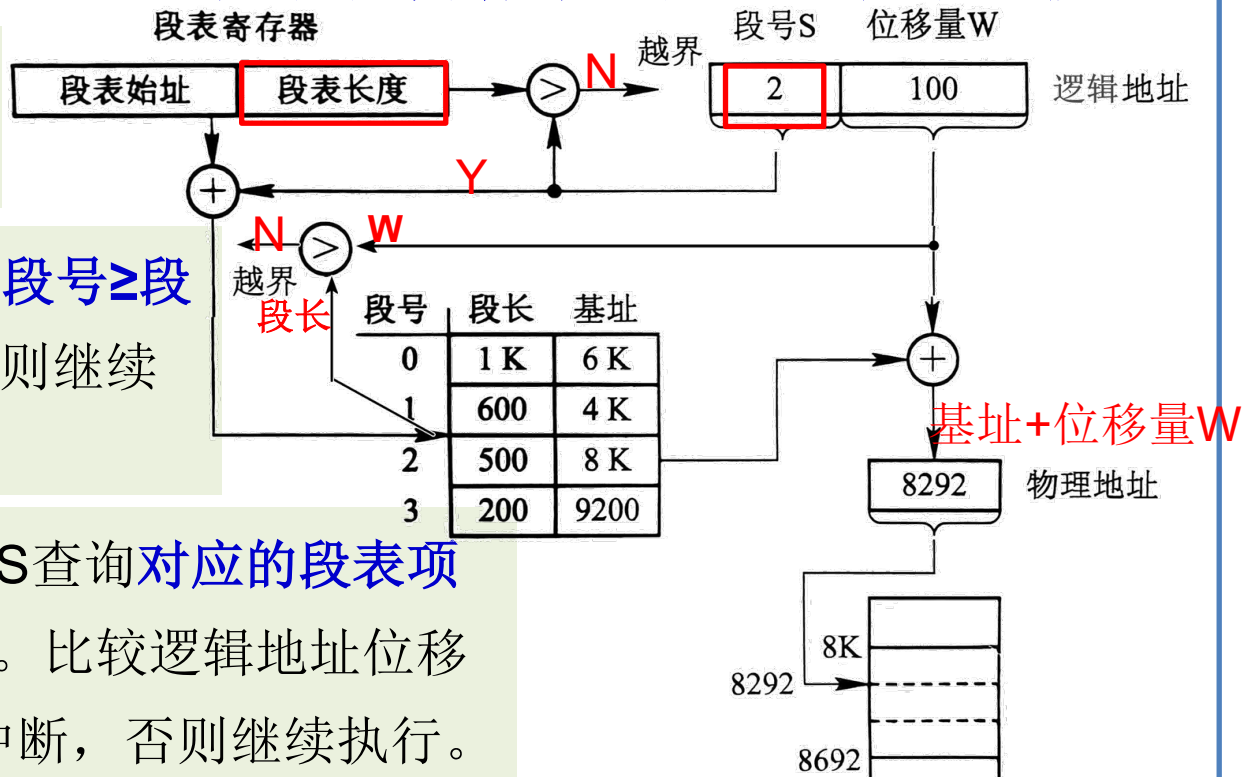
设置一个**段表寄存器**存放**段表在内存中的始址和段表长度**。

1. 将逻辑地址分成段号S和段内偏移量W；

2. 判断段号是否越界：若**段号 $\geq$ 段表长度**，则产生**越界**，否则继续执行；

3. 根据段表始址以及段号S查询**对应的段表项**，确定该段的段长和基址。比较逻辑地址位移量 $W \geq$ 段长，则产生越界中断，否则继续执行。

4. 计算逻辑地址对应的物理地址 = 基址 + 位移量W，访问主存，获取数据。





4.6.2 分段系统的基本原理

4. 分页和分段的主要区别

(1) 信息单位不同

- **分页系统：**页是信息的物理单位。分页是为了实现离散分配，提高内存利用率，是系统行为，对用户不可见；
- **分段系统：**段则是信息的逻辑单位。分段是为了更好的满足用户需求，是用户的行为，对用户可见。



4.6.2 分段系统的基本原理

4. 分页和分段的主要区别

(2) 大小不同

- 分页系统：页的大小固定，由系统决定；
- 分段系统：段的长度不固定，决定于用户编写的程序。

(3) 程序地址空间维数不同

- 分页系统：用户地址空间是一维的，程序员只需给出一个助记符即可表示一个地址；
- 分段系统：用户地址空间是二维的，每段都是从逻辑地址0开始，所以既要给出段名，也要给出段内地址。

分页系统，逻辑上完整的模块物理上不完整，不利于内存信息（数据）共享和保护。



4.6.3 信息共享

1. 分页系统中对程序和数据的共享

在分页系统中实现对程序和数据的共享远不如分段系统方便。

文本编辑程序Text Editor，程序有160KB的代码和另外40KB的数据区。若同时接纳40个用户

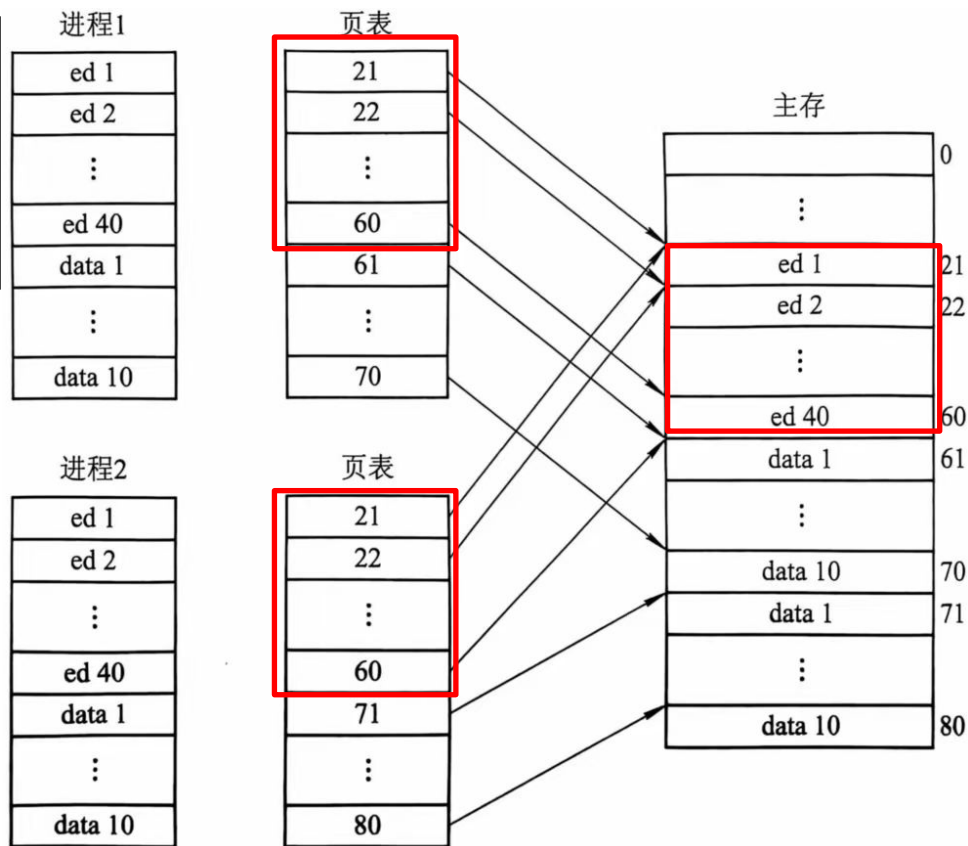
若代码不可重入（不可共享）

——8000KB

代码可重入（可共享）——

1760KB

假定每个页面大小为4KB





4.6.3 信息共享

2. 分段系统中程序和数据的共享

在分段系统中，由于是以段为基本单位的，不管该段有多大，只需为该段设置一个段表项，因此实现共享非常容易。

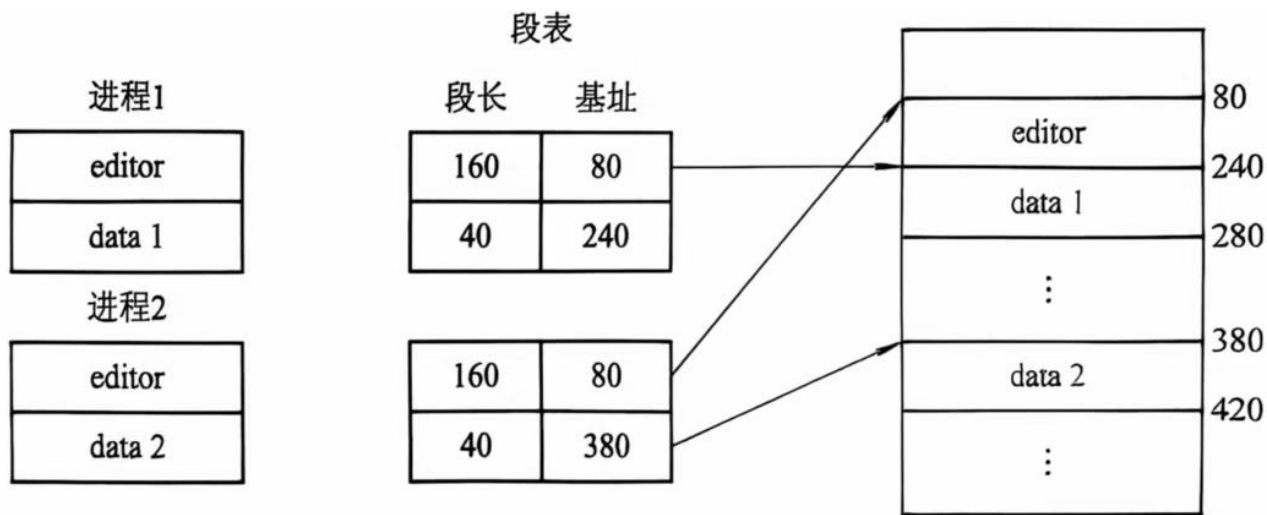


图 4-22 分段系统中共享 editor 的示意图



4.6.4 段页式存储管理方式

分页系统能有效地提高系统内存利用率
分段系统能很好地满足用户的需要

取长补短
段页式存储管理

1. 基本原理

- 内存空间和分页存储管理系统一样，将其分成若干大小相等的物理块。
- 将用户程序分成若干段，再把每段分成若干和物理块大小相同的页。
- 当一个进程被装入物理内存时，系统为该进程的每个段的各页面独立分配一个物理块；一个进程的同一段的多个页面不必存放在连续的多个物理块中。



4.6.4 段页式存储管理方式

1. 基本原理

段页式存储系统中的逻辑地址由3部分组成：段号、段内页号、页内地址。如下所示：

段号S	段内页号P	页内地址W
-----	-------	-------



4.6.4 段页式存储管理方式

利用段表和页表实现地址映射，如图所示。

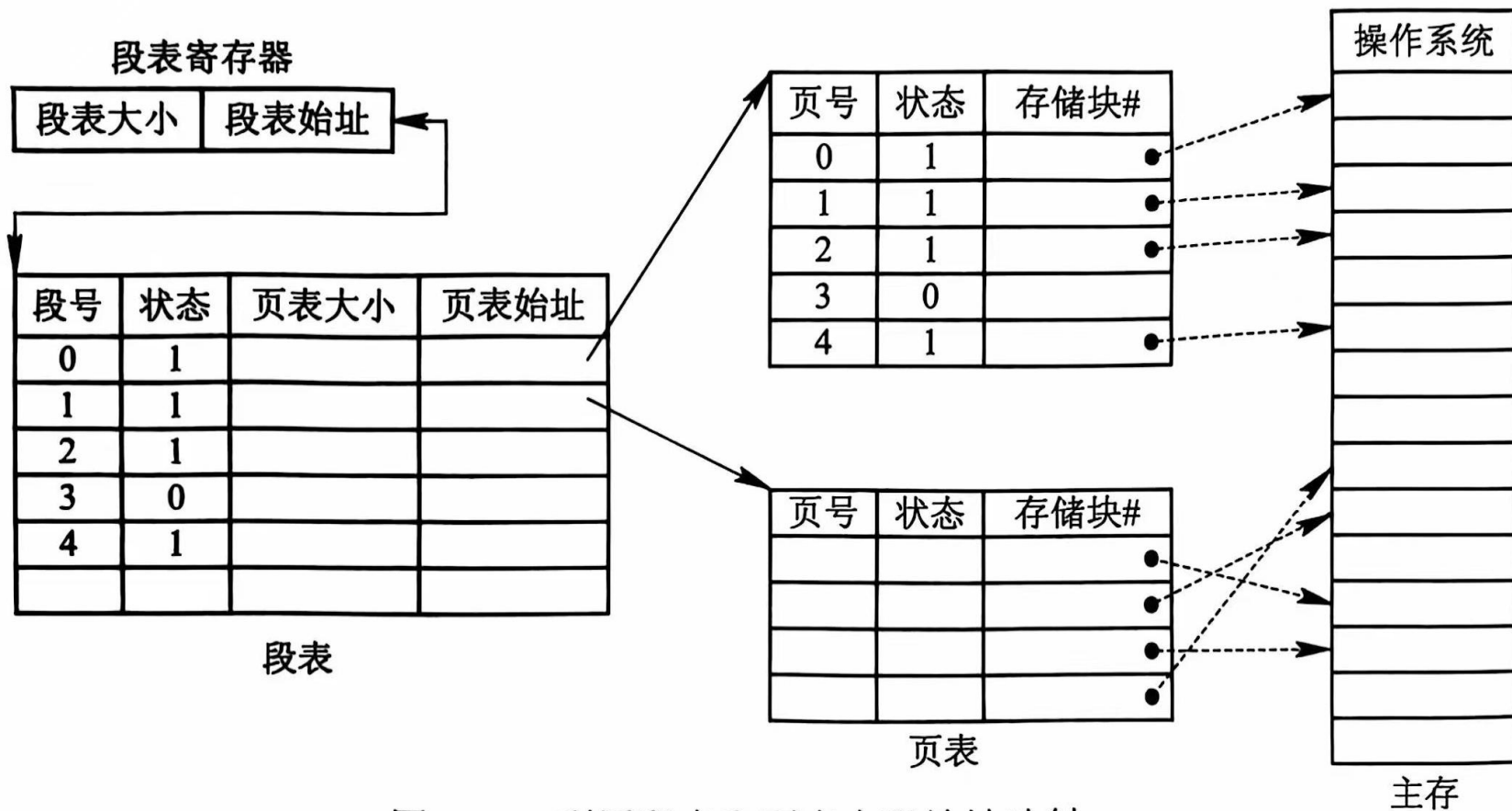


图 4-24 利用段表和页表实现地址映射

访问**3**次内存：第一次，段表；第二次，页表；第三次，取出指令或数据。

4.6.4 段页式存储管理方式

2. 地址变换过程

需配置一个段表寄存器，其中存放段表始址、段表长度。

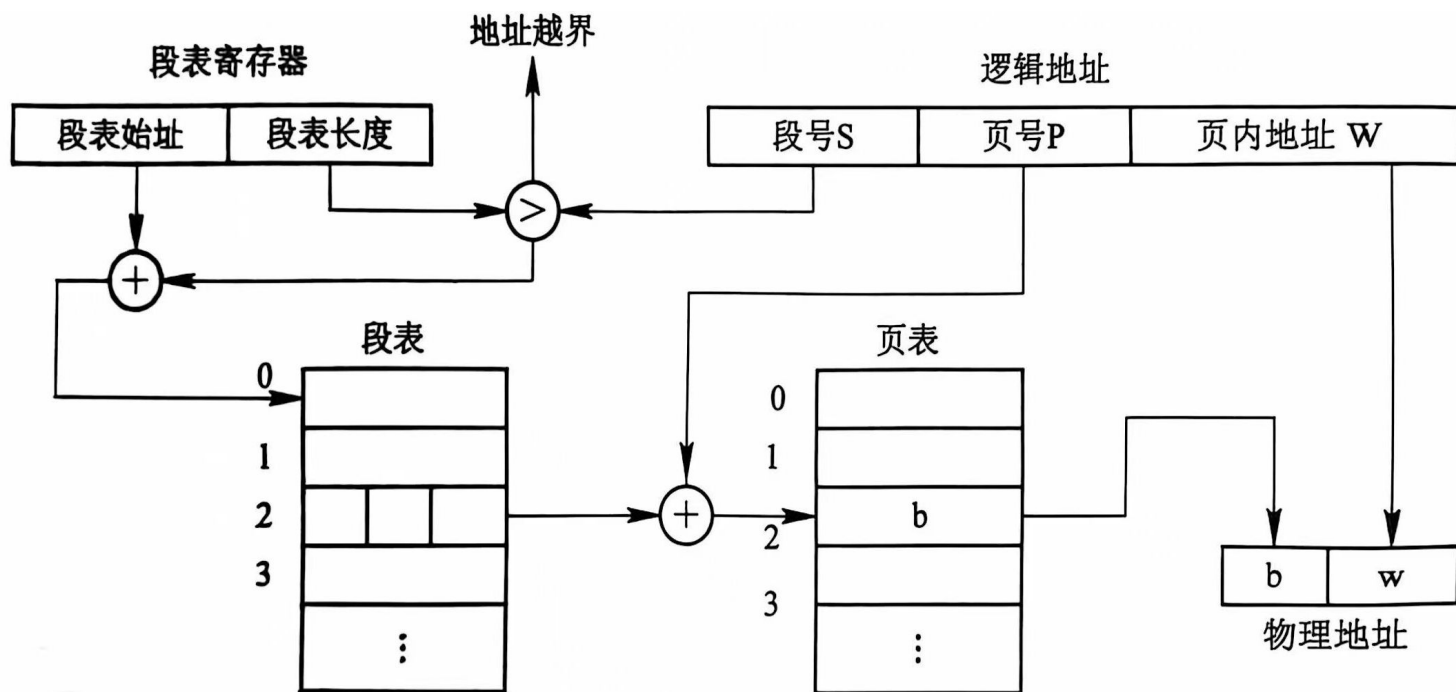


图 4-25 段页式系统中的地址变换机构



河南大學
HENAN UNIVERSITY

明德，新民
止于至善



The End ...

